

Week 4

Optimization & Decision Trees

Nebius Academy



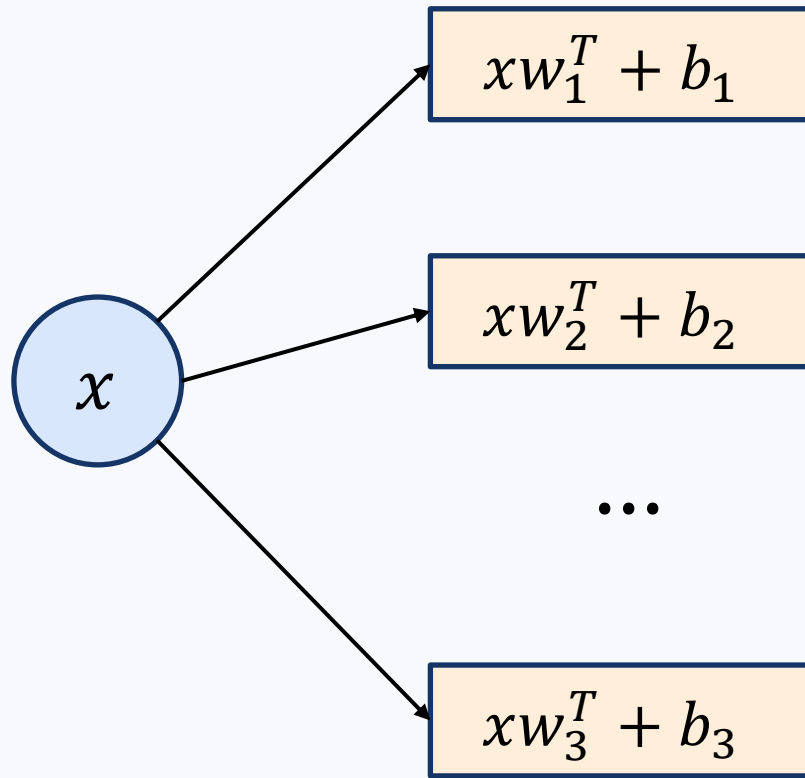
The objectives for today

1. Multiclass classification
2. Gradient optimization
3. Decision trees

Multiclass classification

A model for multiclass classification

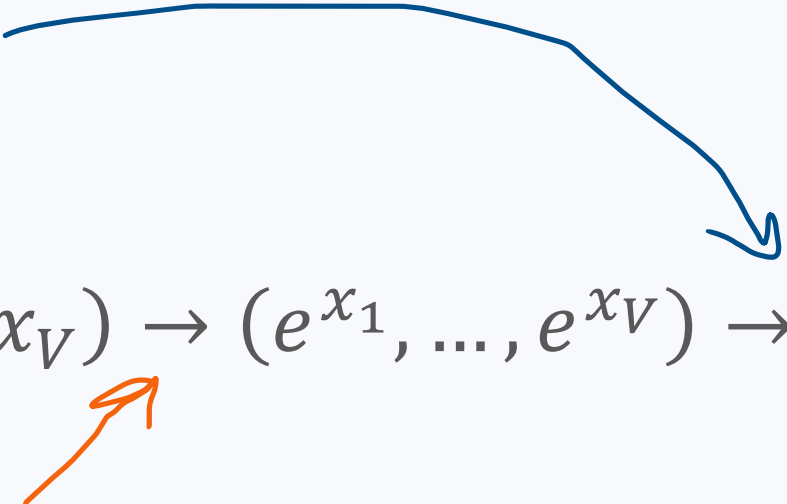
Logits of each class



Softmax

Turns logits (can be any numbers) into probabilities, that

- Are all non-negative,
- Sum to 1

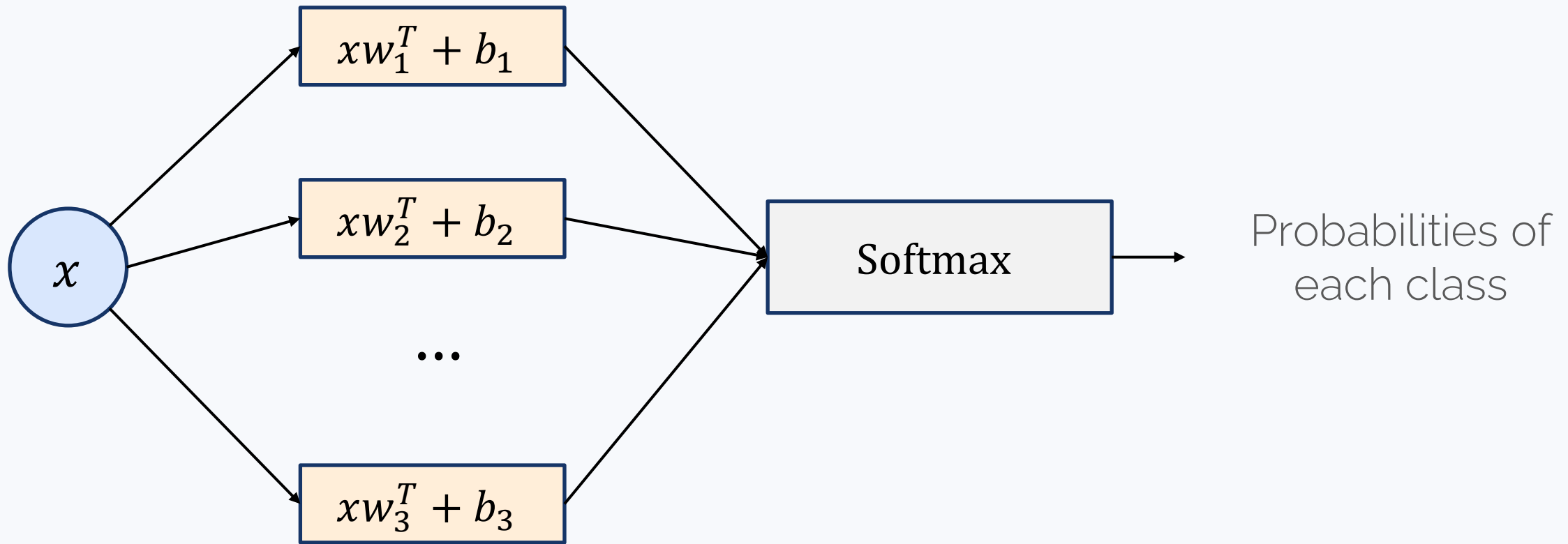

$$(x_1, \dots, x_V) \rightarrow (e^{x_1}, \dots, e^{x_V}) \rightarrow \left(\frac{e^{x_1}}{\sum_t e^{x_t}}, \dots, \frac{e^{x_V}}{\sum_t e^{x_t}} \right)$$

Make non-negative

This is softmax

A model for multiclass classification

Logits of each class



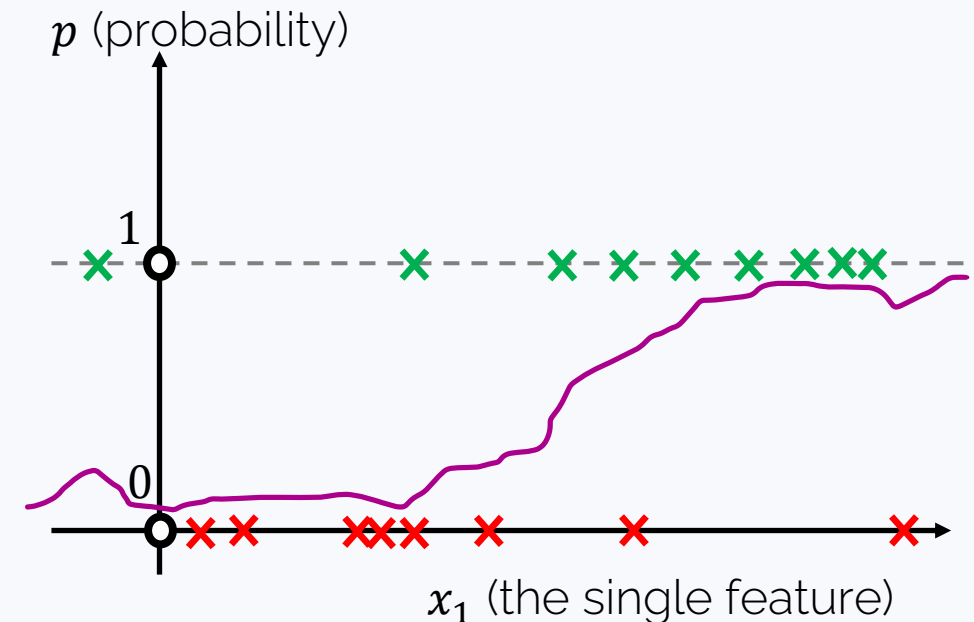
Cross-entropy loss

We **minimize**:

$$l(y_i, \hat{p}_i) = \begin{cases} -\log \hat{p}_i, & \text{if } y_i = 1 \\ -\log(1 - \hat{p}_i), & \text{if } y_i = 0 \end{cases}$$

y_i — true class labels
 $\hat{p}_i$ — predicted probabilities

$$\mathcal{L}(y_i, \hat{p}_i) = -y_i \log \hat{p}_i - (1 - y_i) \log(1 - \hat{p}_i)$$



Cross-entropy loss, >2 classes

We **minimize**:

$$l(y_i, \hat{p}_i) = -\log \hat{p}_{i,y_i}$$

y_i — true class labels

$\hat{p}_{i,y_i}$ — predicted
probability of class y_i

$$\mathcal{L}(y_i, \hat{p}_i) = -\sum_{k=1}^K [y_i = k] \cdot \log \hat{p}_{ik}$$

Cross-entropy loss, >2 classes

We **minimize**:

$$l(y_i, \hat{p}_i) = -\log \hat{p}_{i,y_i}$$

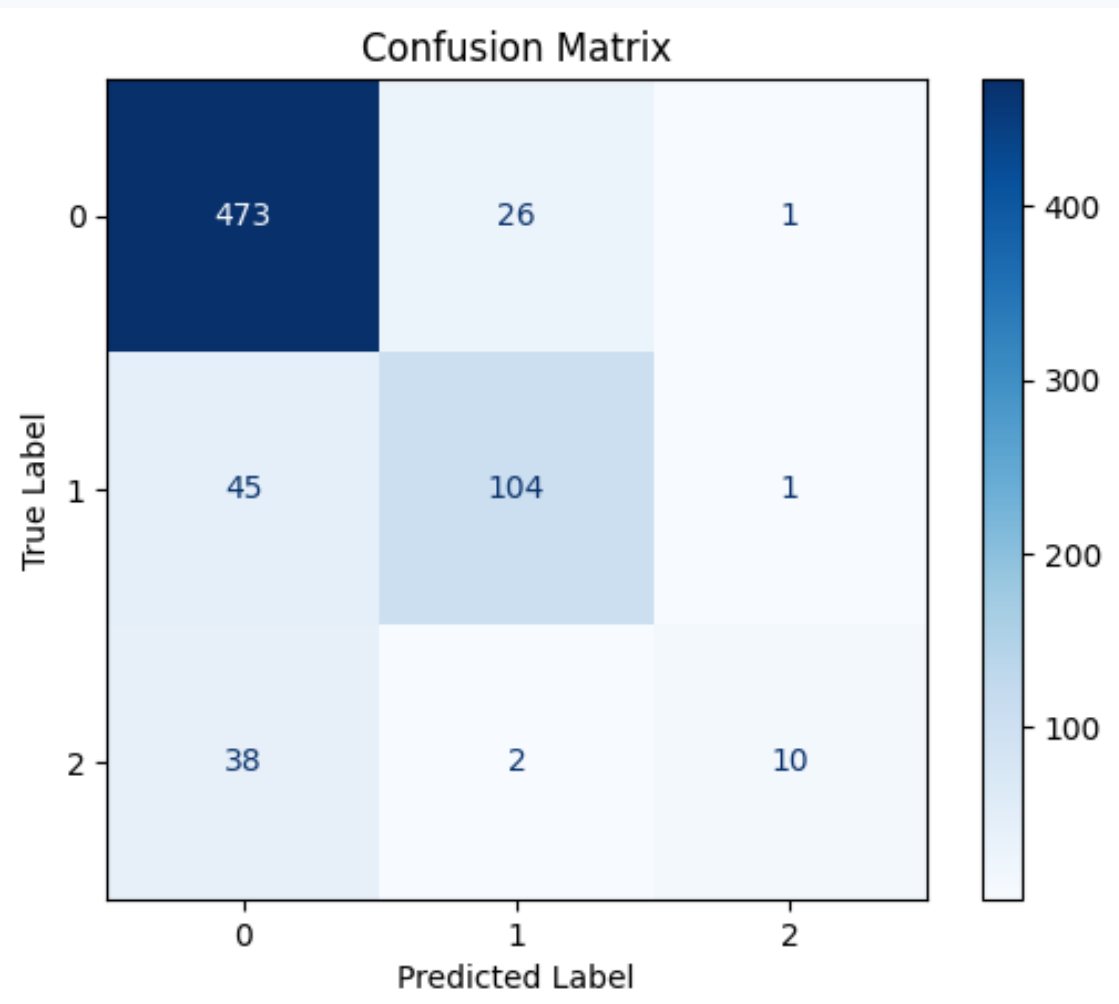
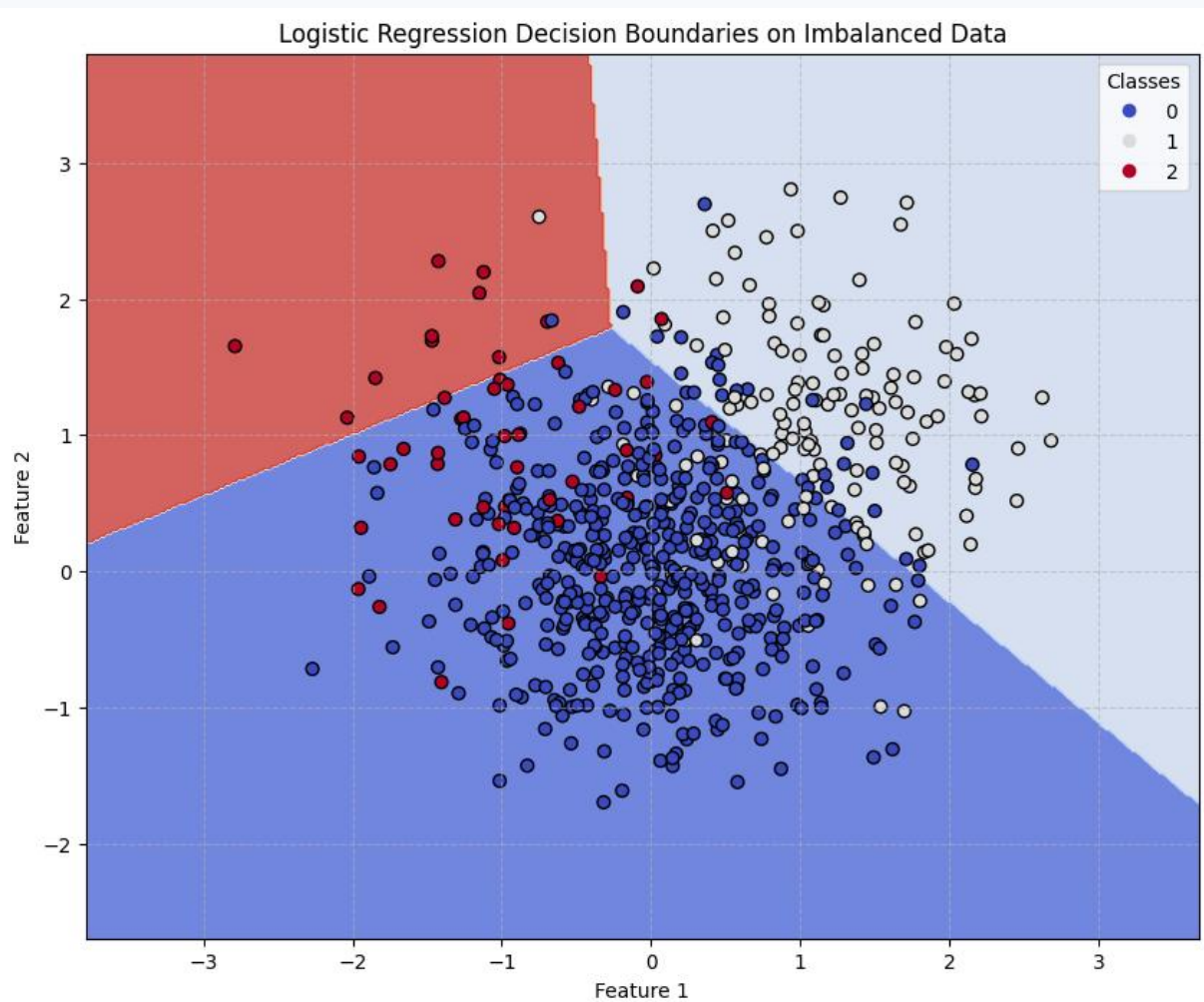
y_i – true class labels

$\hat{p}_{i,y_i}$ – predicted
probability of class y_i

$$\mathcal{L}(y_i, \hat{p}_i) = - \sum_{k=1}^K [y_i = k] \cdot \log \hat{p}_{ik}$$

$$\mathcal{L}(y, \hat{p}) = - \sum_{(x_i, y_i)} \sum_{k=1}^K [y_i = k] \cdot \log \hat{p}_{ik}$$

Confusion matrix



Micro and macro averaging

Construct K classifiers: k -th class vs the rest

For each, compute TP_k , TN_k , FP_k , FN_k .

- **Micro averaging:** first average ($TP = \frac{1}{K} \sum_k TP_k$), then compute precision and recall
- **Macro averaging:** average resulting precisions and recalls

Gradient optimization

A quick reminder about loss functions

Regression loss

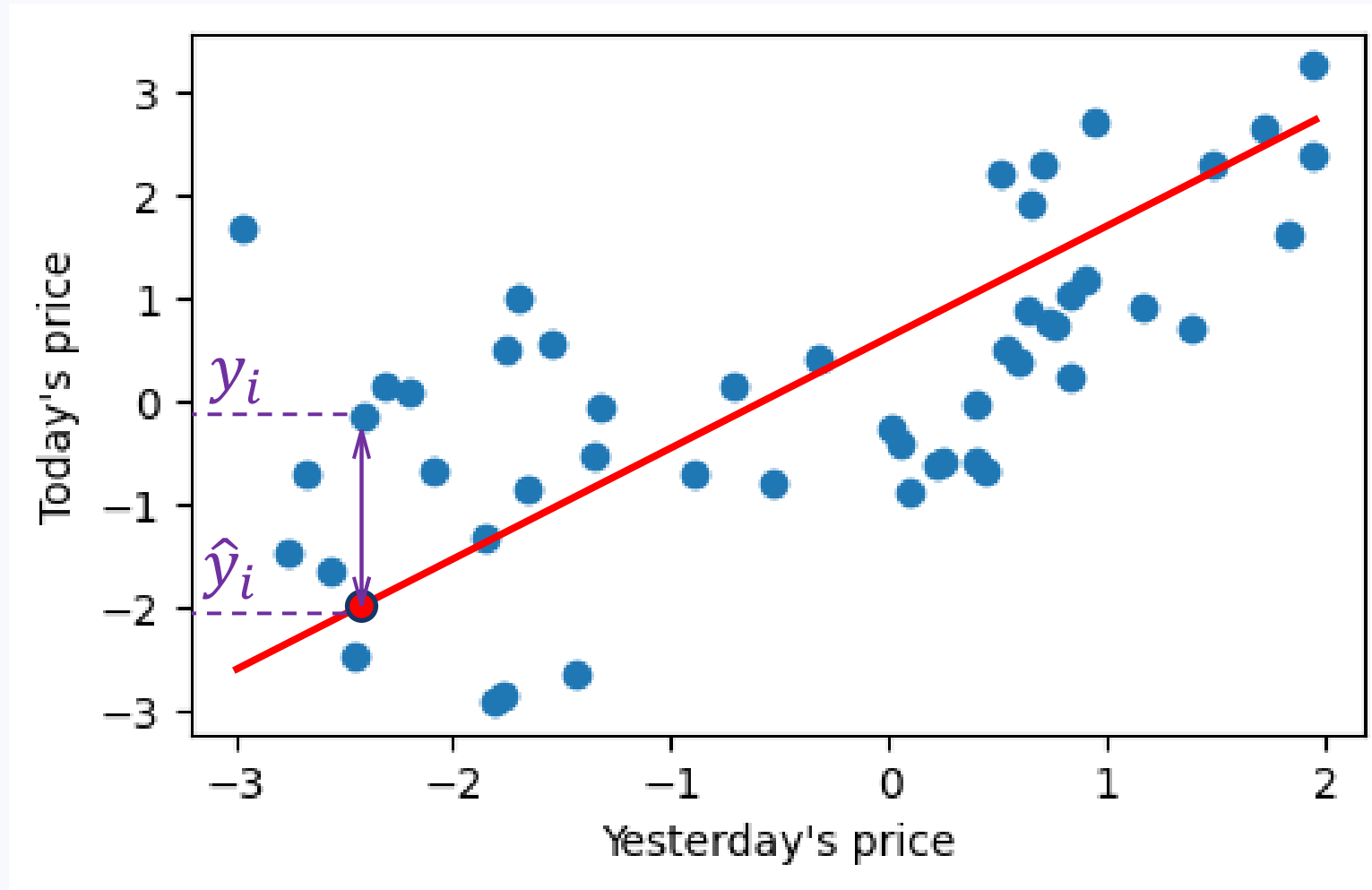
$$\mathcal{L}(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y_i, \hat{y}_i)$$

y_i — true value

$\hat{y}_i$ — predicted value

MSE:

$$\mathcal{L}(y_i, \hat{y}_i) = (y_i - \hat{y}_i)^2$$



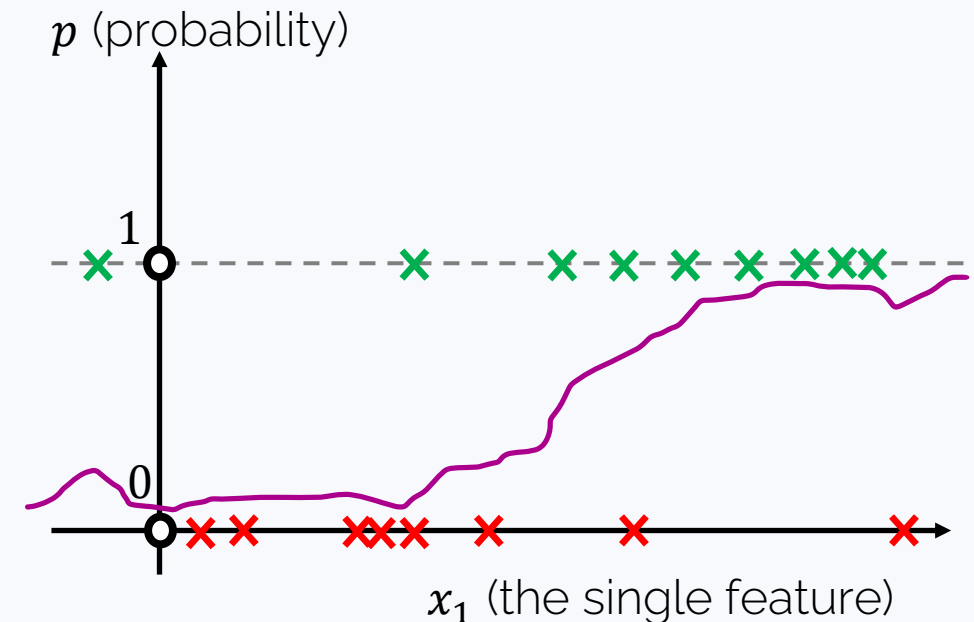
A quick reminder about loss functions

Classification: cross-entropy

$$\mathcal{L}(y_i, \hat{p}_i) = -y_i \log \hat{p}_i - (1 - y_i) \log(1 - \hat{p}_i)$$

$$l(y_i, \hat{p}_i) = \begin{cases} -\log(1 - \hat{p}_i), & \text{if } y_i = 0 \\ -\log \hat{p}_i, & \text{if } y_i = 1 \end{cases}$$

y_i — true class labels
 $\hat{p}_i$ — predicted probabilities



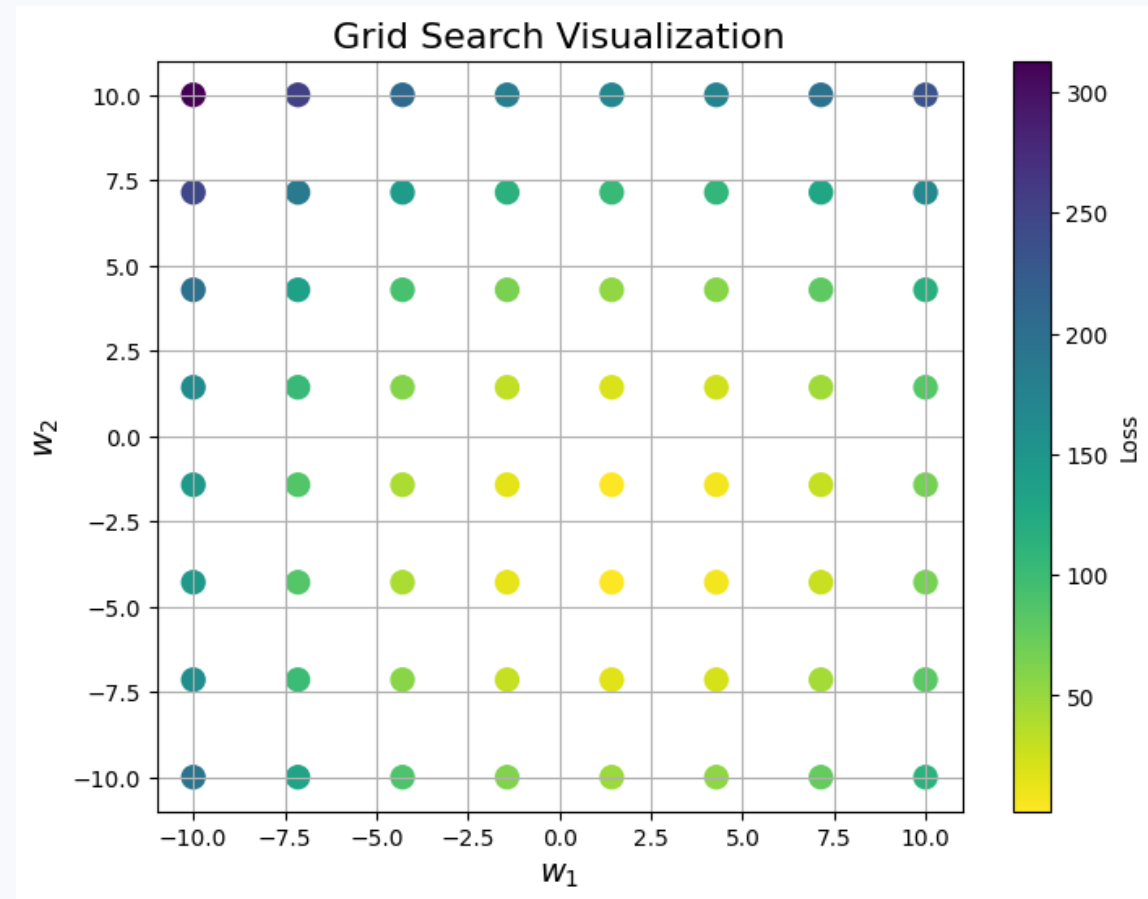
The next step – how to find the optimal w

We need to find the parameters $\mathbf{w}$, which minimize the loss.

The next step – how to find the optimal w

We need to find the parameters w , which minimize the loss.

Why not grid search?



The next step – how to find the optimal w

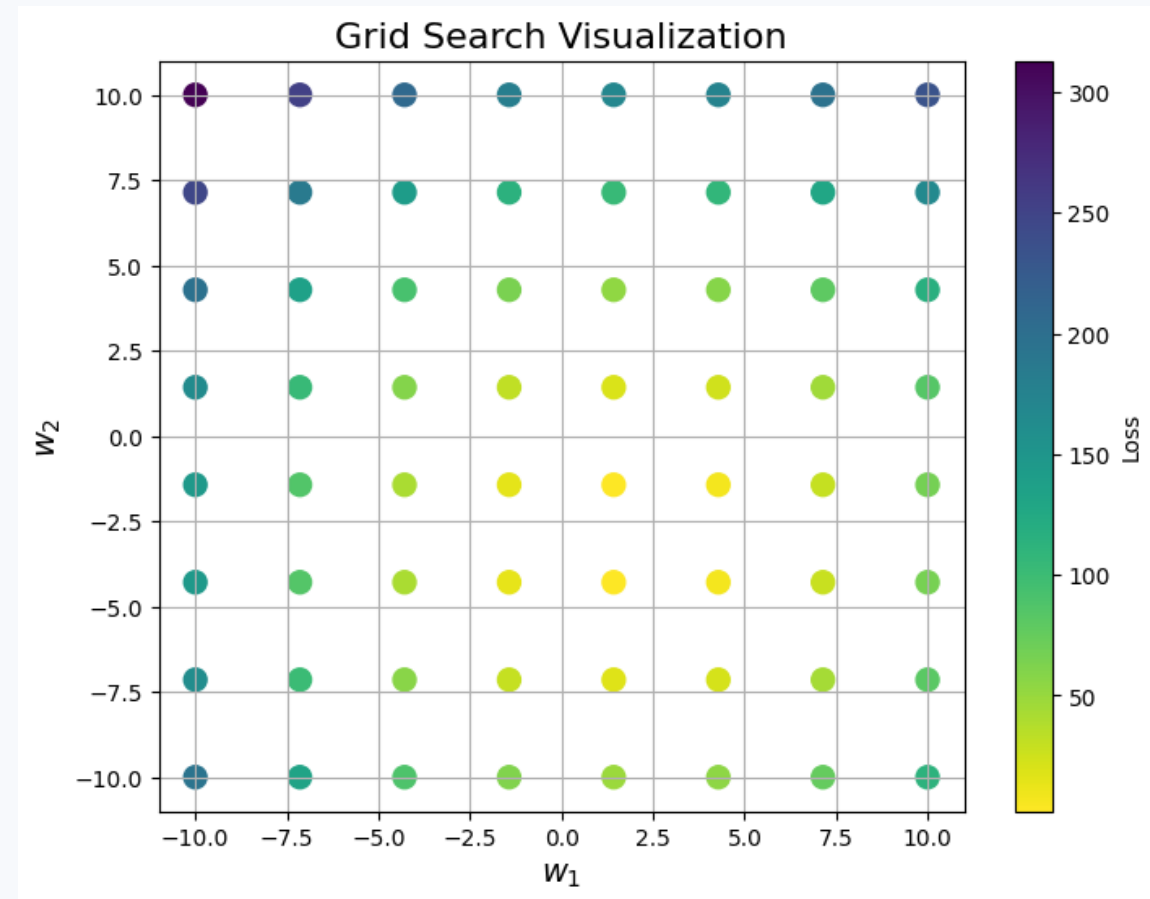
We need to find the parameters w , which minimize the loss.

Why not grid search?

Imagine:

- 20 parameters
- -2..2 with 0.1 step (41 values)
- Training set size: 200
- Test set size: 100

We need to calculate $\mathcal{L}$ many times:
 $41^{20} \cdot (200 + 100)$

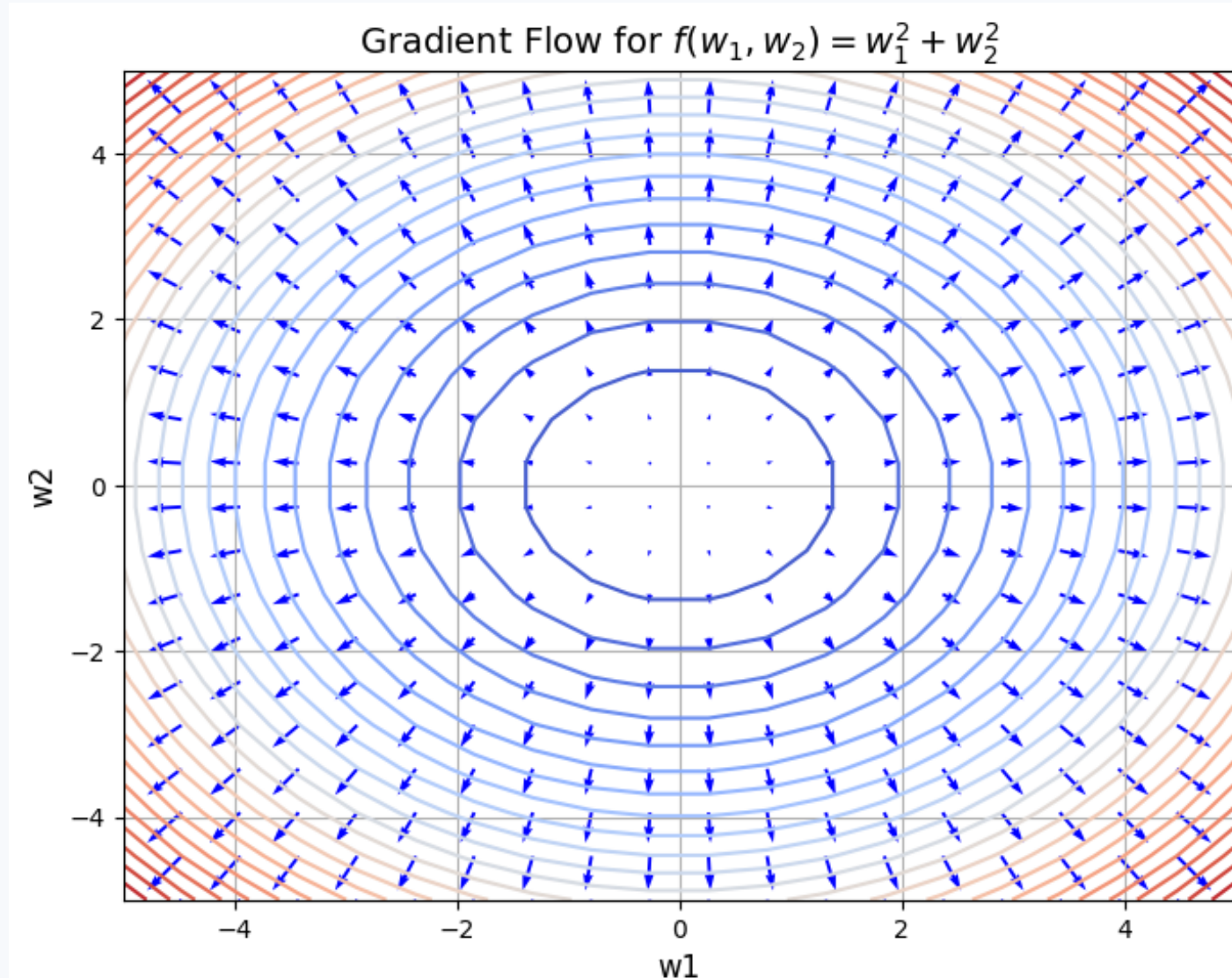


Gradients

What is a gradient

Gradient of $f(w_1, \dots, w_D)$ at a point $w_0 = (w_{01}, \dots, w_{0D})$ is the vector

$$\begin{aligned} & [\nabla_w f](w_0) \\ &= \left(\frac{\partial f}{\partial w_1}, \dots, \frac{\partial f}{\partial w_D} \right) \bigg|_{w=w_0} \end{aligned}$$

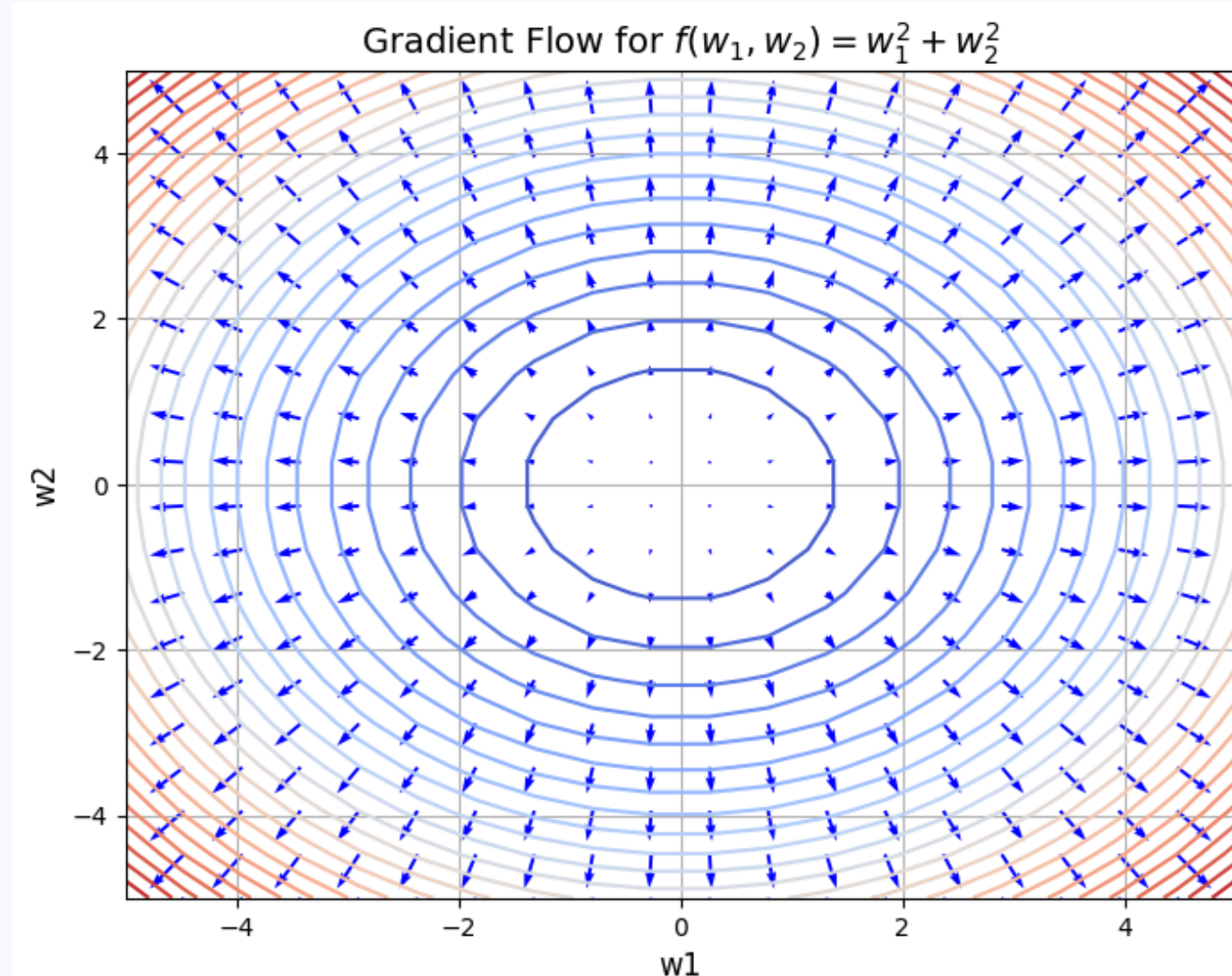


What is a gradient

Gradient of $f(w_1, \dots, w_D)$ at a point $w_0 = (w_{01}, \dots, w_{0D})$ is the vector

$$\begin{aligned} & [\nabla_w f](w_0) \\ &= \left(\frac{\partial f}{\partial w_1}, \dots, \frac{\partial f}{\partial w_D} \right) \bigg|_{w=w_0} \end{aligned}$$

It is the **direction in which the function increases the fastest**



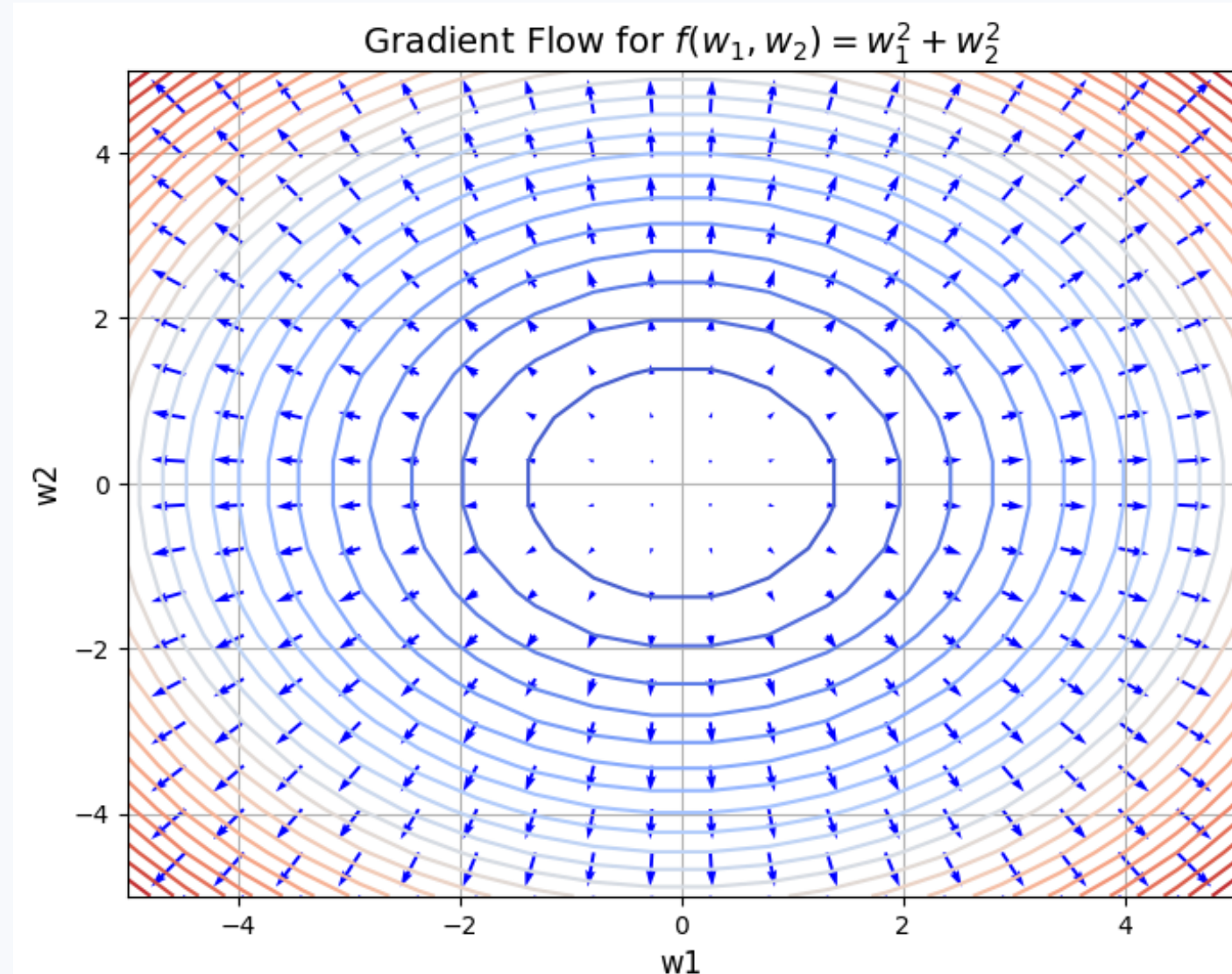
An obvious application

Loss minimum is where the gradient is zero

Solve

$$\frac{\partial f}{\partial w_1} = \dots = \frac{\partial f}{\partial w_D} = 0,$$

get the answer.



Linear regression

How it works

Loss minimum is where the gradient is zero

Solve

$$\frac{\partial f}{\partial w_1} = \dots = \frac{\partial f}{\partial w_D} = 0,$$

get the answer.

$$\begin{aligned}\mathcal{L}(X, w, y) &= \sum_{i=1}^N (y_i - x_i w^T)^2 \\ &= \sum_{i=1}^N \left(y_i - \sum_{j=1}^D x_{ij} w_j \right)^2\end{aligned}$$

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial w_j} &= \sum_{i=1}^N 2(y_i - x_i w^T) \cdot (-x_{ij}) = \\ &= -2 \sum_{i=1}^N (y - Xw^T)_i \cdot x_{ij} \\ &= \left[-2(y - Xw^T)^T X \right]_j\end{aligned}$$

$$\text{Solution: } w^T = (X^T X)^{-1} X^T y$$

How it works

Loss minimum is where the gradient is zero

Solve

$$\frac{\partial f}{\partial w_1} = \dots = \frac{\partial f}{\partial w_D} = 0,$$

get the answer.

Linear regression

$$X = QR$$

(Q with orthogonal columns; R square)

Solution:

$$\begin{aligned} w^T &= (X^T X)^{-1} X^T y = \\ &= (R^T \textcolor{red}{Q}^T \textcolor{red}{Q} R)^{-1} R^T Q^T y = \\ &= (R^T R)^{-1} R^T Q^T y = \\ &= R^{-1} R^{-T} R^T Q^T y = R^{-1} (Q^T y) \end{aligned}$$

Logistic regression

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\begin{aligned}\sigma'(z) &= \frac{1}{(1 + e^{-z})^2} (e^{-z}) = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z} + 1 - 1}{1 + e^{-z}} = \\ &= \sigma(z)(1 - \sigma(z))\end{aligned}$$

Logistic regression

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\begin{aligned}\sigma'(z) &= \frac{1}{(1 + e^{-z})^2} (e^{-z}) = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z} + 1 - 1}{1 + e^{-z}} = \\ &= \sigma(z)(1 - \sigma(z))\end{aligned}$$

$$\log \sigma(z)' = \frac{1}{\sigma(z)} \sigma(z)(1 - \sigma(z)) = 1 - \sigma(z)$$

$$\log(1 - \sigma(z))' = \frac{1}{1 - \sigma} \cdot -\sigma(1 - \sigma) = -\sigma(z)$$

Logistic regression

$$\mathcal{L}(X, w, y) = - \sum_{i=1}^N (1 - y_i) \log \left(1 - \sigma \left(\sum_{j=1}^D x_{ij} w_j \right) \right) + y_i \log \sigma \left(\sum_{j=1}^D x_{ij} w_j \right) =$$

Logistic regression

$$\mathcal{L}(X, w, y) = - \sum_{i=1}^N (1 - y_i) \log \left(1 - \sigma \left(\sum_{j=1}^D x_{ij} w_j \right) \right) + y_i \log \sigma \left(\sum_{j=1}^D x_{ij} w_j \right) =$$

$$\frac{\partial \mathcal{L}}{\partial w_j} = - \sum_{i=1}^N (1 - y_i) \cdot -\sigma(x_i w^T) x_{ij} + y_i \left(1 - \sigma(x_j w^T) \right) x_{ij} =$$

Logistic regression

$$\mathcal{L}(X, w, y) = - \sum_{i=1}^N (1 - y_i) \log \left(1 - \sigma \left(\sum_{j=1}^D x_{ij} w_j \right) \right) + y_i \log \sigma \left(\sum_{j=1}^D x_{ij} w_j \right) =$$

$$\frac{\partial \mathcal{L}}{\partial w_j} = - \sum_{i=1}^N (1 - y_i) \cdot -\sigma(x_i w^T) x_{ij} + y_i (1 - \sigma(x_i w^T)) x_{ij} =$$

$$= - \sum_{i=1}^N [-\sigma(x_i w^T) + y_i (\sigma - \sigma + 1)] x_{ij} = - \sum_{i=1}^N [y_i - \sigma(x_i w^T)] x_{ij} =$$

$$= - (y - \sigma(Xw^T))^T X$$

Compare:
 $-2(y - Xw^T)^T X$

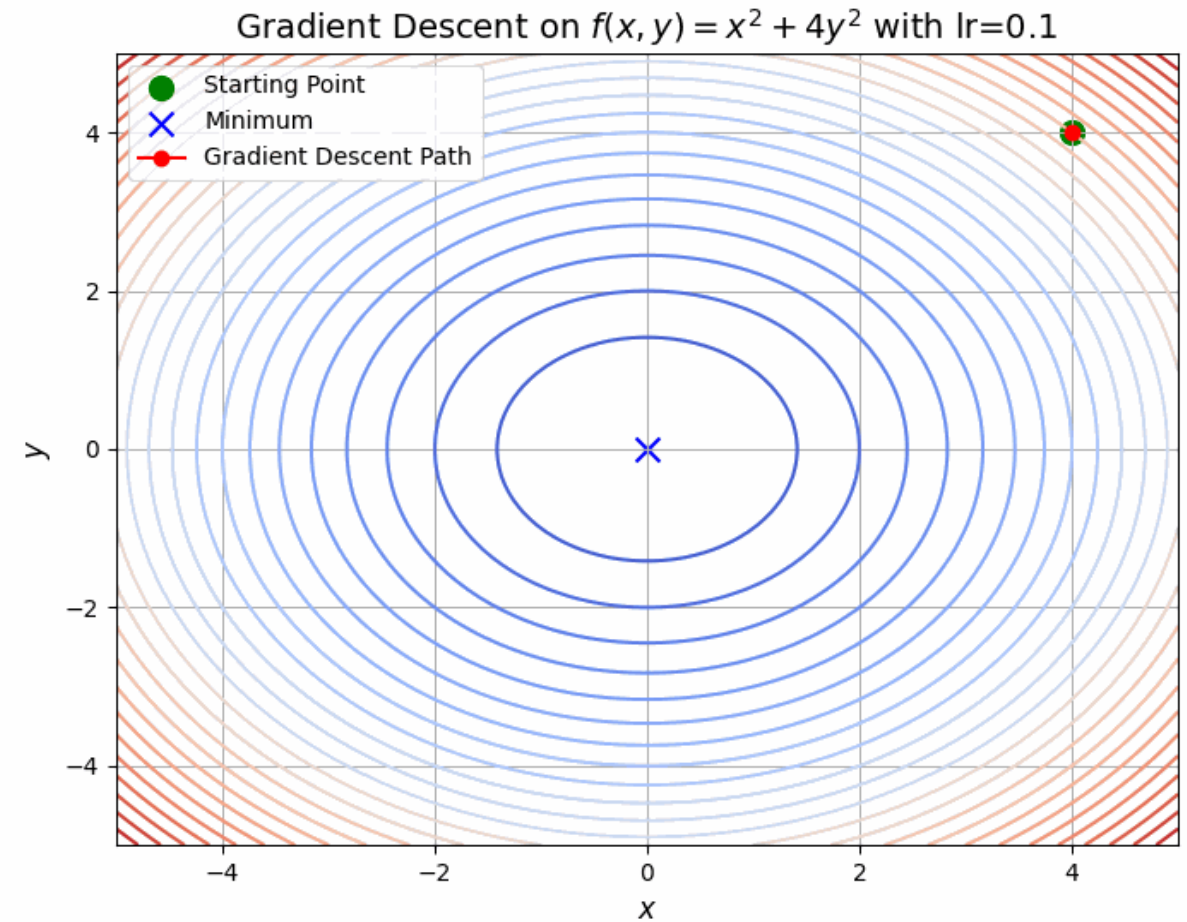
We can't solve it ☹

Gradient descent

Gradient is the direction of the steepest increase $\Rightarrow$

Anti-gradient $-\nabla f$ is the direction of the steepest decrease

Just go along the anti-gradient to reach the minimum!

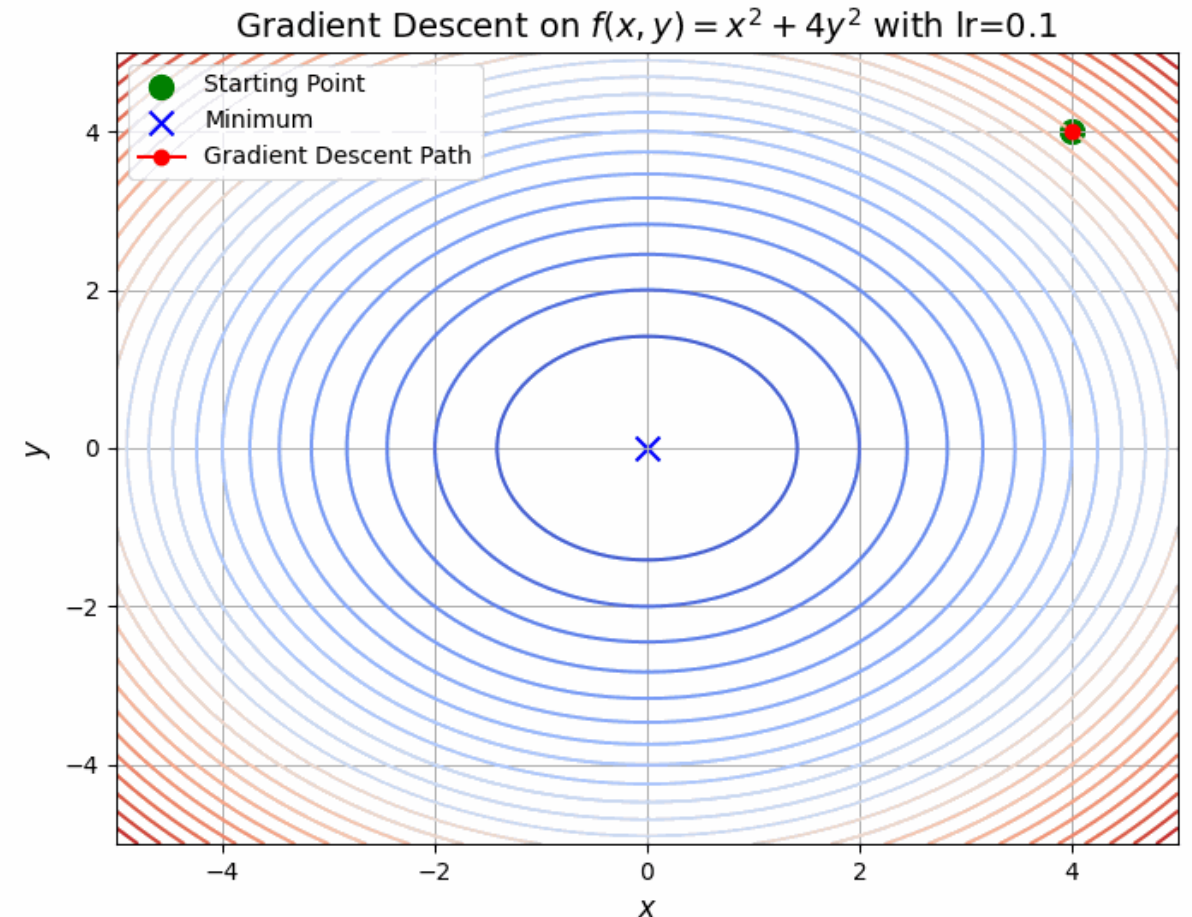


Gradient descent

Math formulation:

Given: f , ∇f , w_0 , **max_steps**,
 α – learning rate

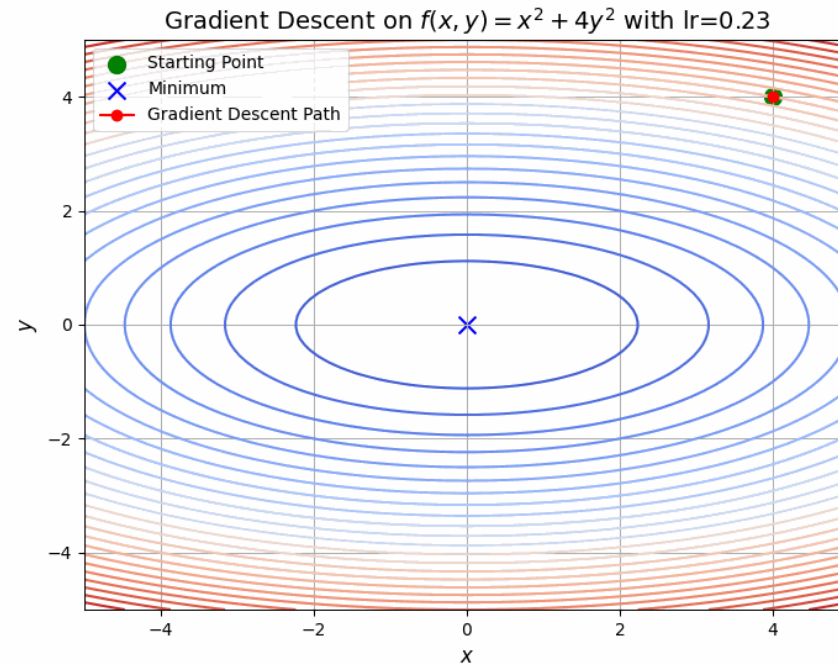
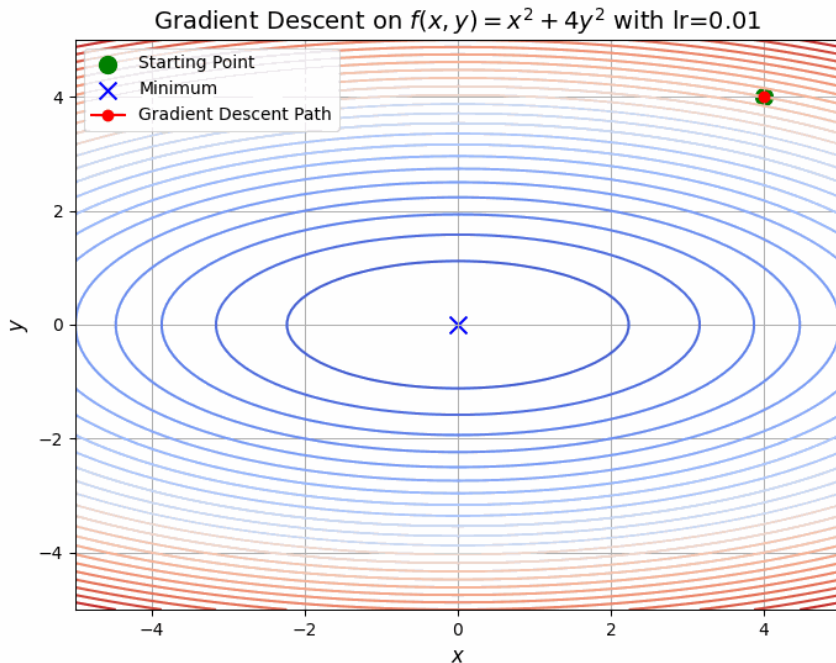
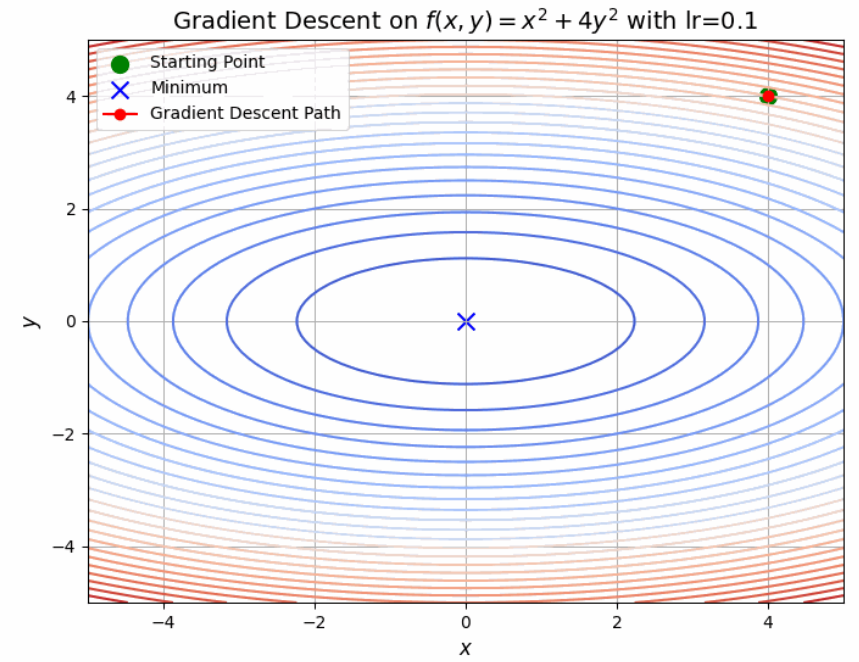
1. Initialize w with w_0
2. Perform several anti-gradient steps:
$$w_{m+1} = w_m - \alpha \cdot \nabla_w f(w_m)$$
3. Finish when reached **max_steps** or converged



Learning rate α is important

If it's too little, it won't converge;

If it's too big, it will diverge.



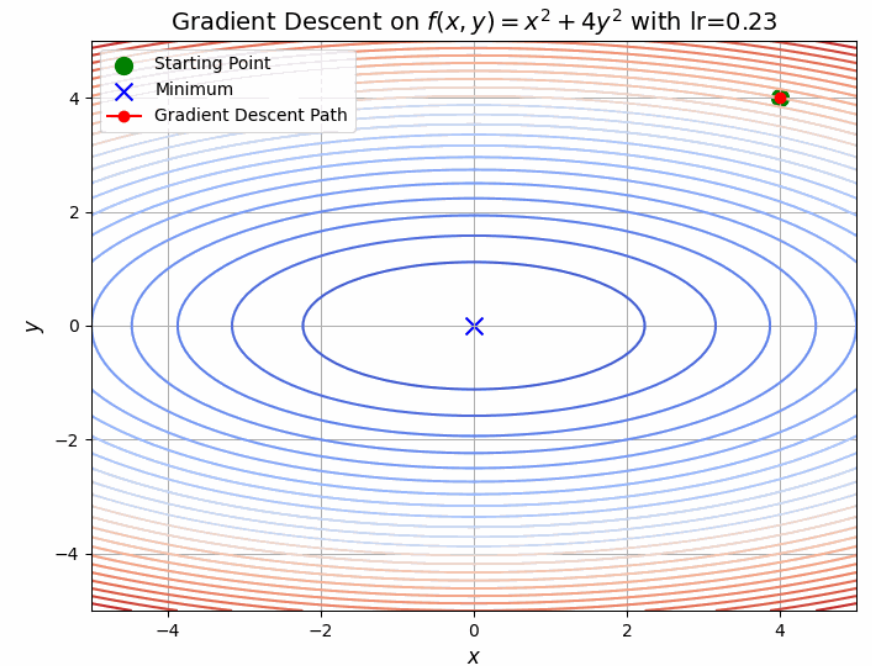
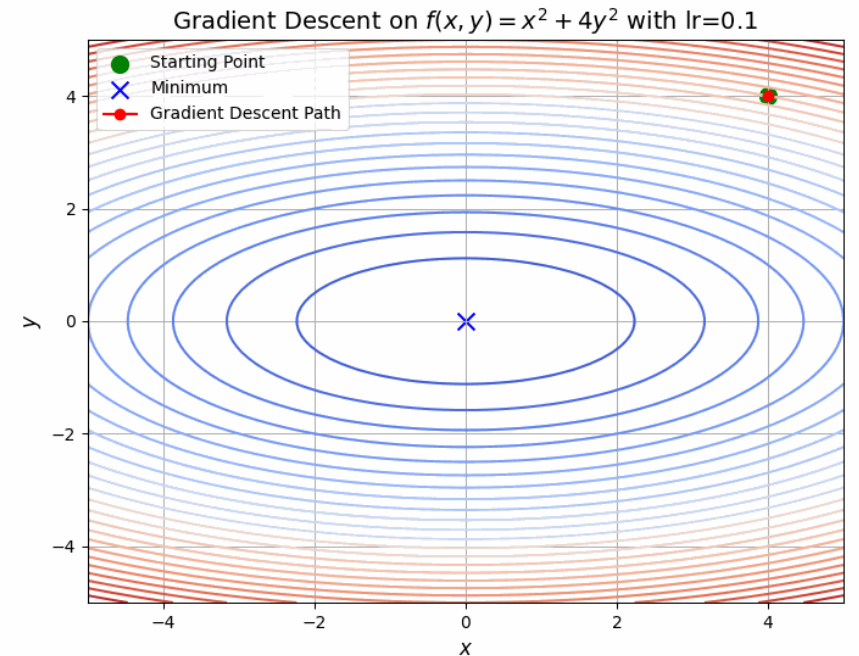
Learning rate is important

If it's too little, it won't converge;

If it's too big, it will diverge.

Learning rate is a **hyperparameter** of optimization tuned on a logarithmic scale. The choice would be between: $1e-5$, $1e-4$, $1e-3$, $1e-2$

In practice, learning rate decay is often used.



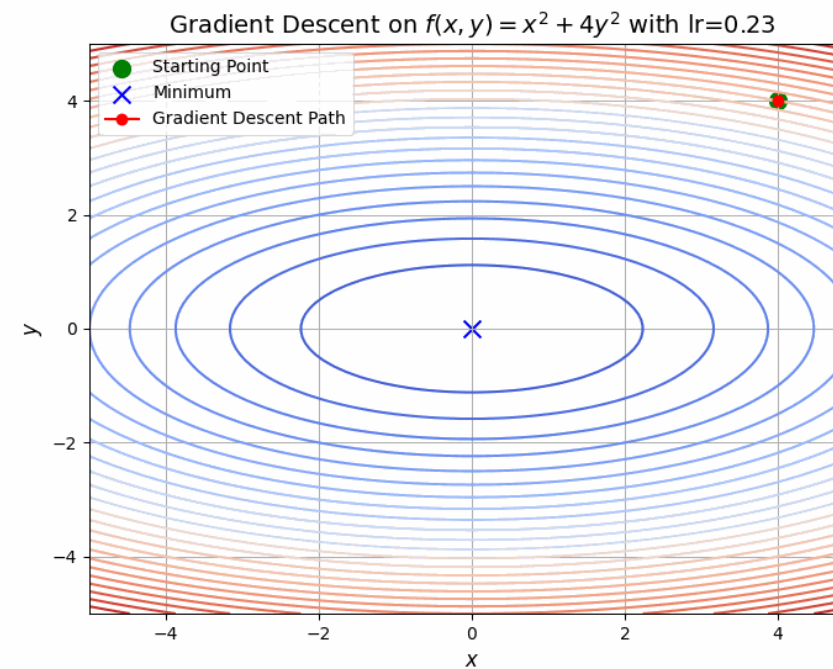
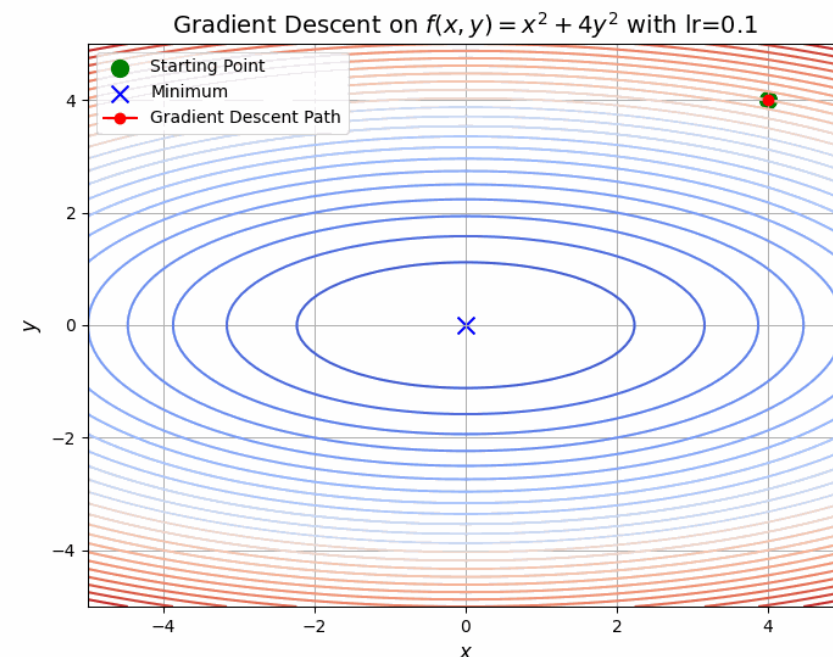
About elongated loss basin

$$\mathcal{L}(X, w, y) = \sum_{i=1}^N (y_i - x_i w^T)^2 =$$

$$= \|y - Xw^T\|^2 = (y - Xw^T)^T (y - Xw^T) =$$

$$= (y^T - wX^T)(y - Xw^T) =$$

$$= wX^T Xw^T + \text{linear terms}$$



About elongated loss basin

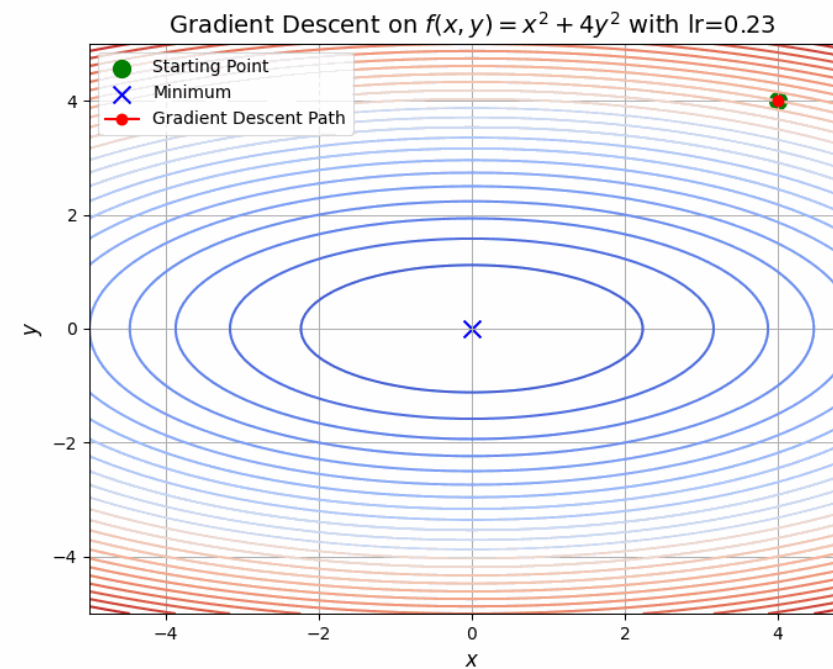
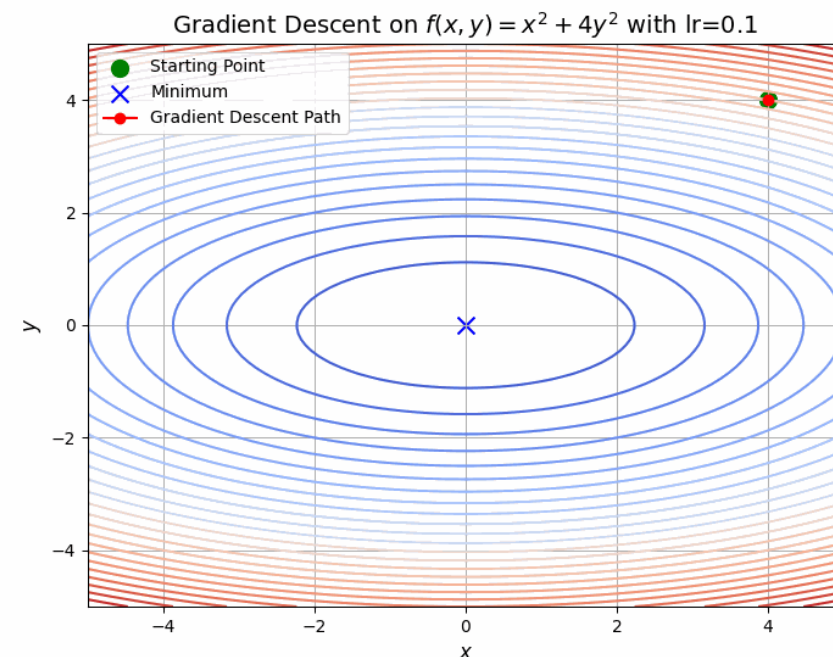
$$\mathcal{L}(X, w, y) = \sum_{i=1}^N (y_i - x_i w^T)^2 =$$

$$= wX^T X w^T + \text{linear terms}$$

If every feature has *mean* = 0, then

$$(X^T X)_{ij} \sim \text{Cov}(\text{feature}_i, \text{feature}_j)$$

$$(X^T X)_{ii} \sim \mathbb{V}(\text{feature}_i)$$



About elongated loss basin

$$(X^T X)_{ij} =$$

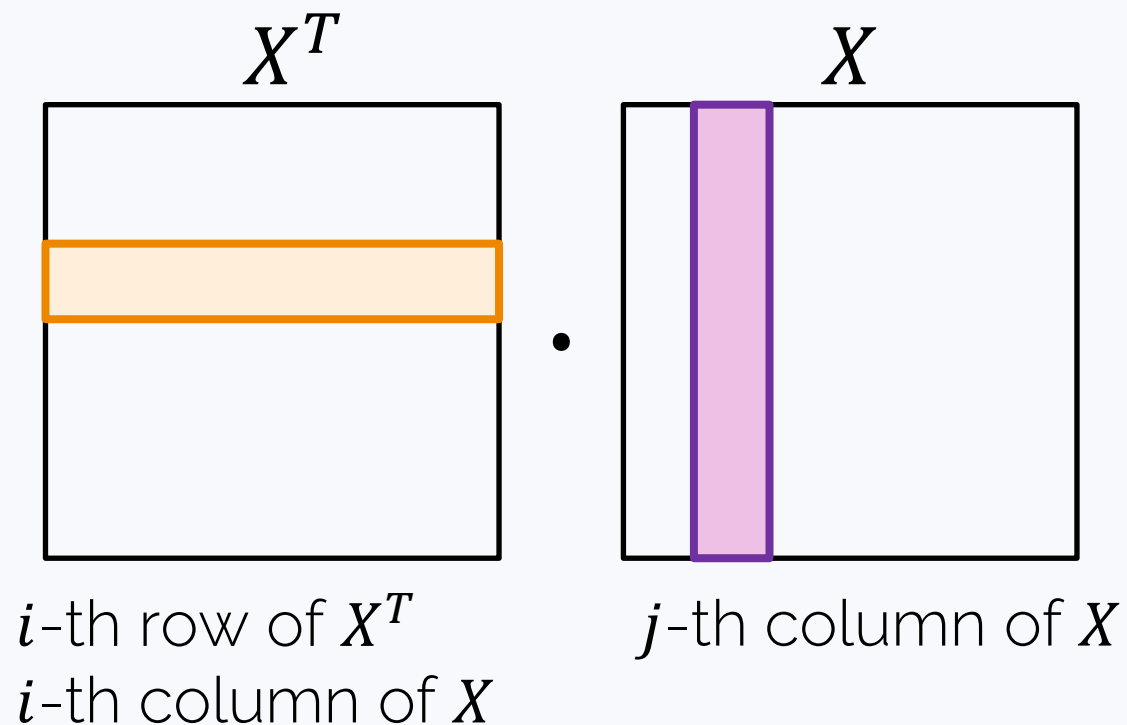
$$\mathcal{L}(X, w, y) = \sum_{i=1}^N (y_i - x_i w^T)^2 =$$

$$= w X^T X w^T + \text{linear terms}$$

If every feature has *mean* = 0, then

$$(X^T X)_{ij} \sim \text{Cov}(\text{feature}_i, \text{feature}_j)$$

$$(X^T X)_{ii} \sim \mathbb{V}(\text{feature}_i)$$



About elongated loss basin

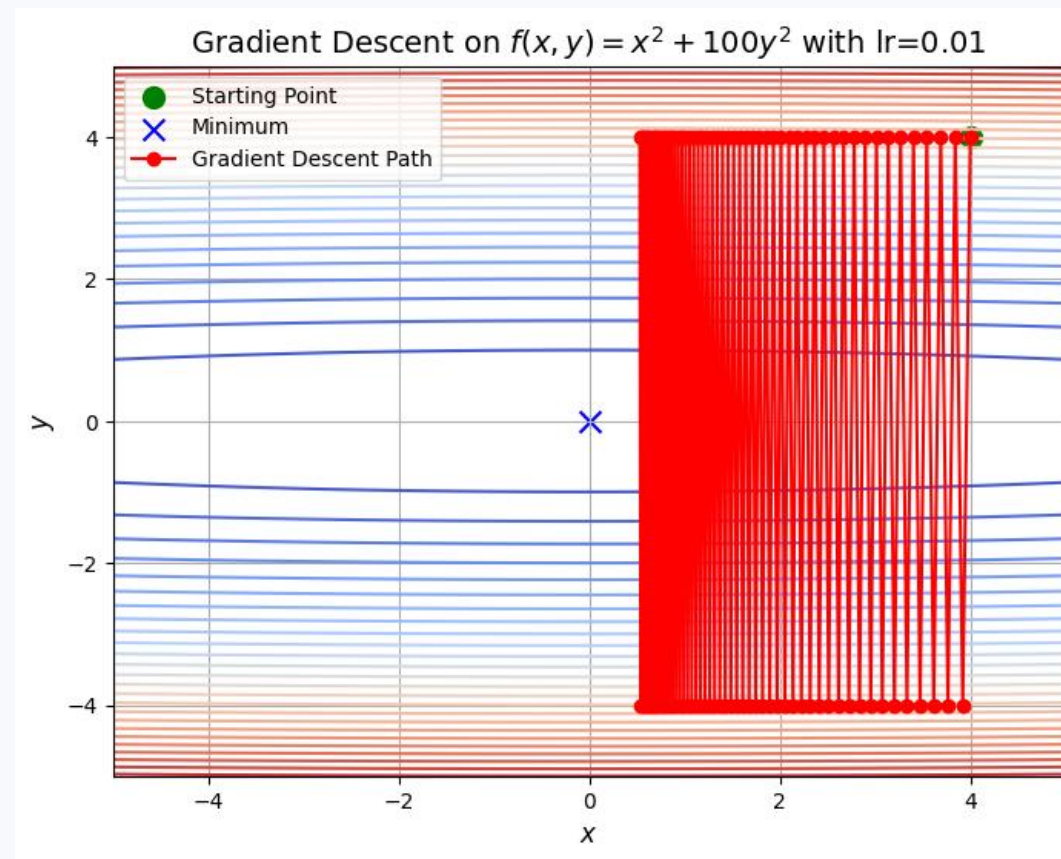
$$\mathcal{L}(X, w, y) = \sum_{i=1}^N (y_i - x_i w^T)^2 =$$

$$= wX^T X w^T + \text{linear terms}$$

If every feature has *mean* = 0, then

$$(X^T X)_{ij} \sim \text{Cov}(\text{feature}_i, \text{feature}_j)$$

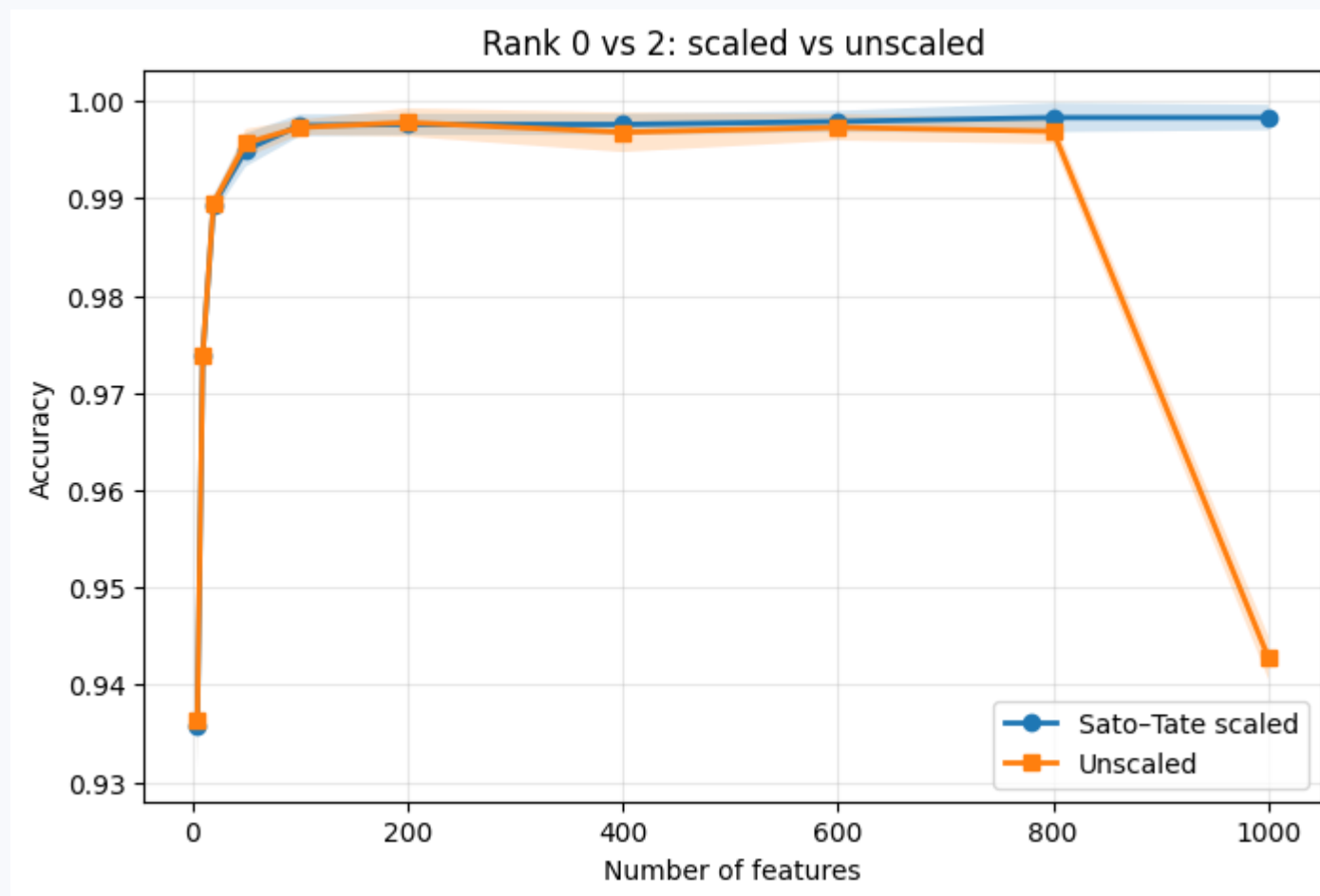
$$(X^T X)_{ii} \sim \mathbb{V}(\text{feature}_i)$$



That's why feature normalization matters

Feature scaling:

$$a_p \rightarrow \frac{a_p}{2\sqrt{p}}$$

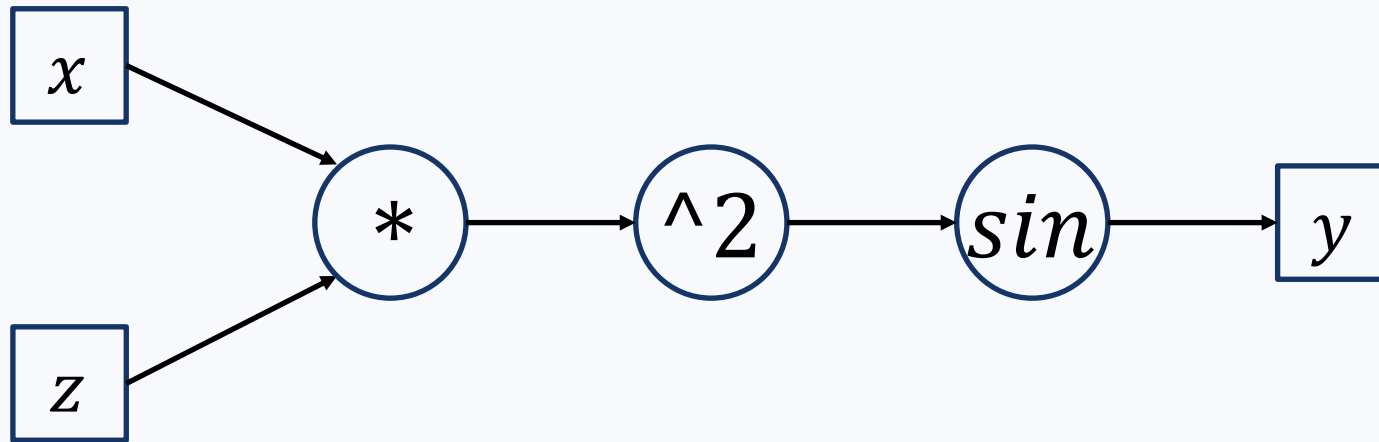


Autodiff

Automating differentiation

Every expression is a computational graph

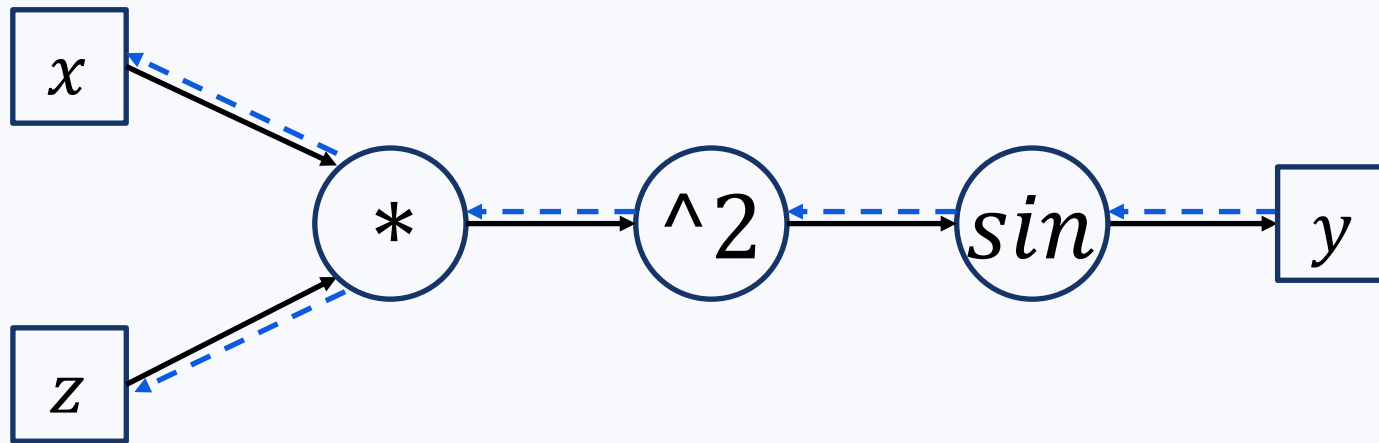
Example. $f(x, z) = \sin(x \cdot z)^2$



Automating differentiation

Every expression is a computational graph

Example. $f(x, z) = \sin(x \cdot z)^2$

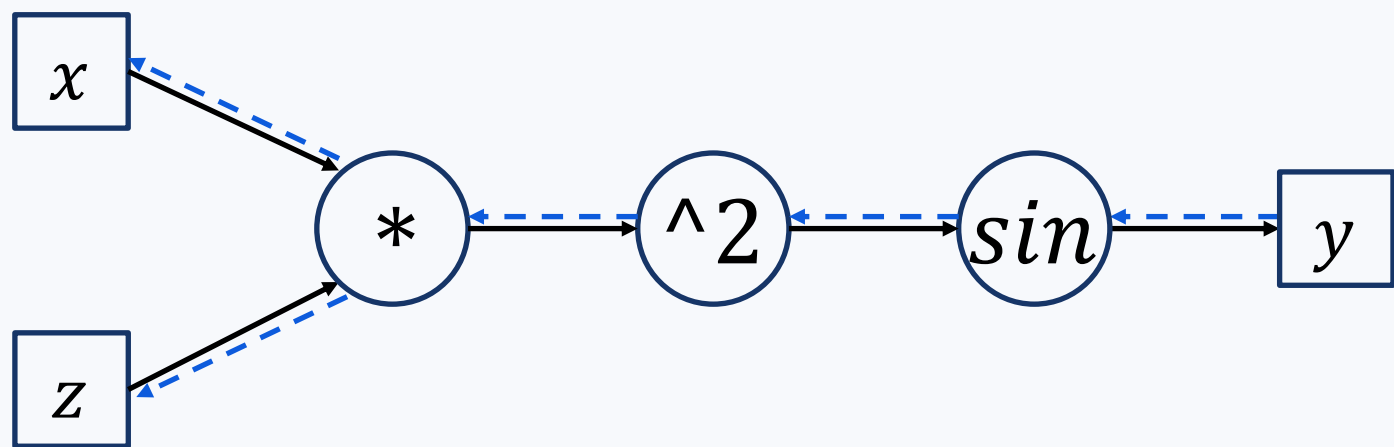


Derivatives flow backward

Automating differentiation

Every expression is a computational graph

Example. $f(x, z) = \sin(x \cdot z)^2$



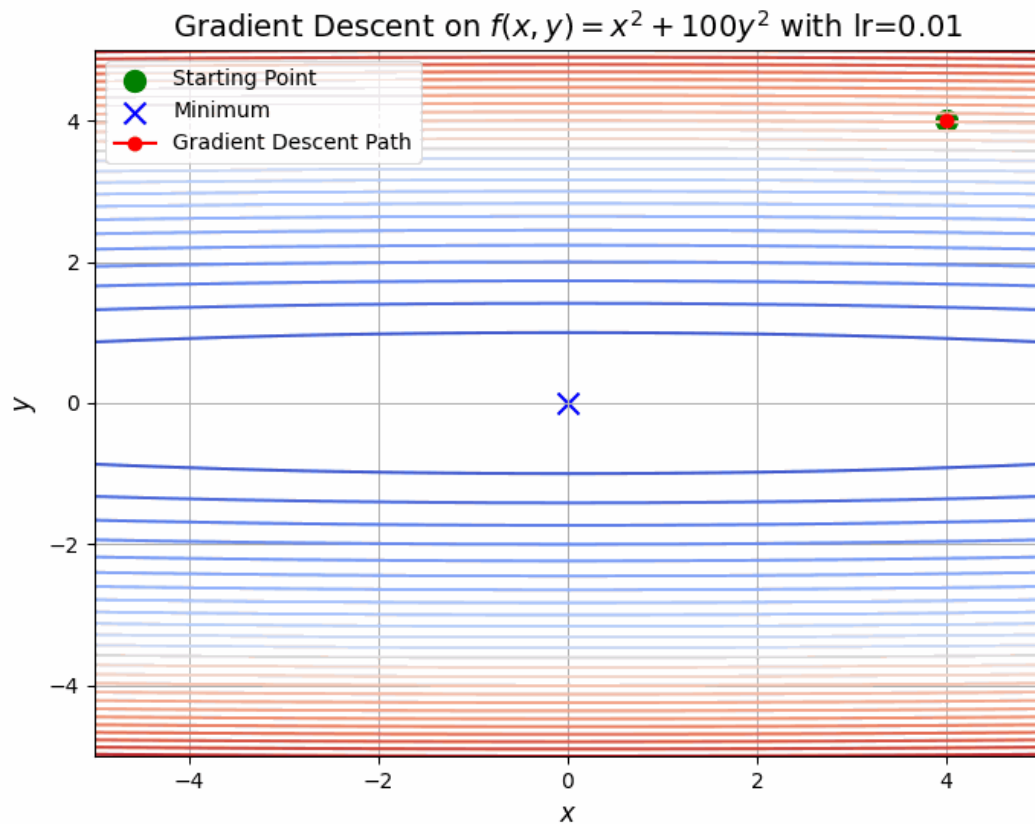
Derivatives flow backward

$$\begin{aligned} & \left. \frac{\partial}{\partial x} \sin(x \cdot z)^2 \right|_{\substack{x=1 \\ z=2}} \\ &= \cos \left((x \cdot z)^2 \Big|_{\substack{x=1 \\ z=2}} \right) \\ & \cdot 2 \left((x \cdot z) \Big|_{\substack{x=1 \\ z=2}} \right) \\ & \cdot z \end{aligned}$$

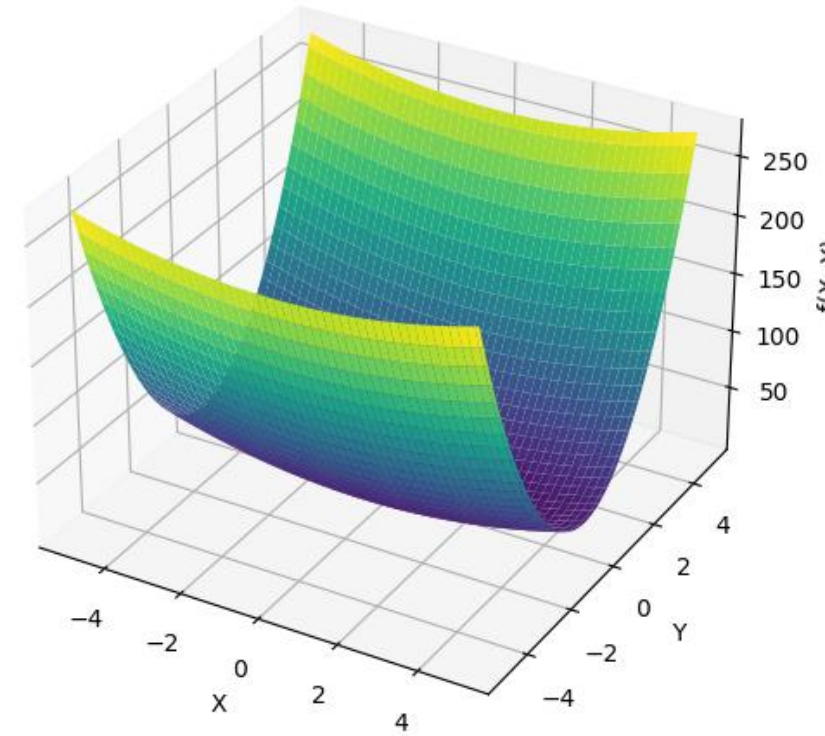
Why gradient
optimization is tricky?

Badly conditioned problems

Very steep gradients are bad



3D plot of $f(x, y) = x^2 + 10y^2$



Having slopes of very different magnitude along different directions is bad

Badly conditioned problems

Very steep gradients are bad

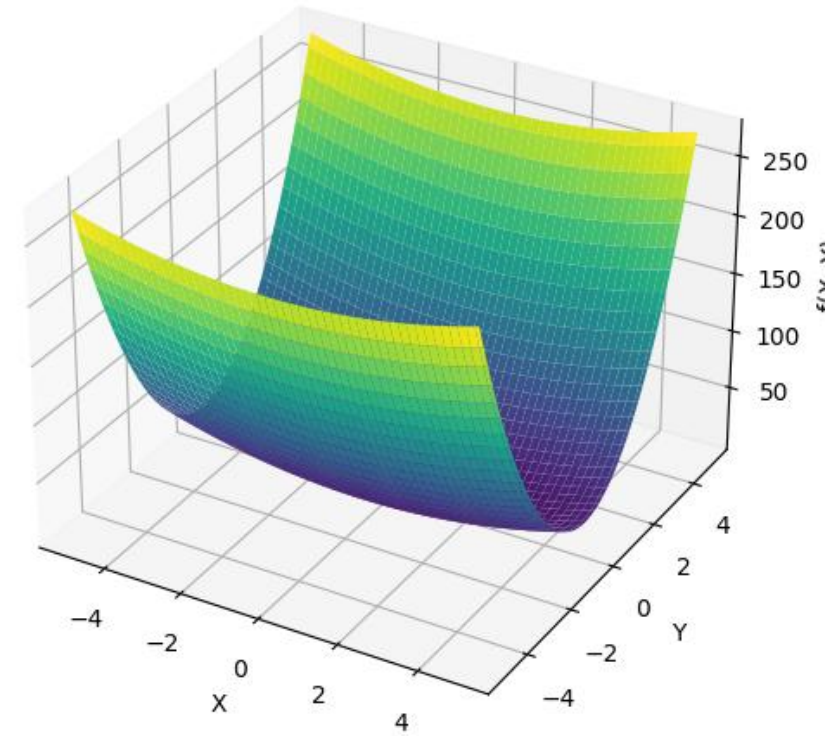
A bit of math:

- For a smooth and strongly convex f after k steps the precision ($|f(x_k) - f(x_*)|$) will be

$$O\left(\min\left\{R^2 \exp\left(-\frac{k}{4\kappa}\right), \frac{R^2}{k}\right\}\right)$$

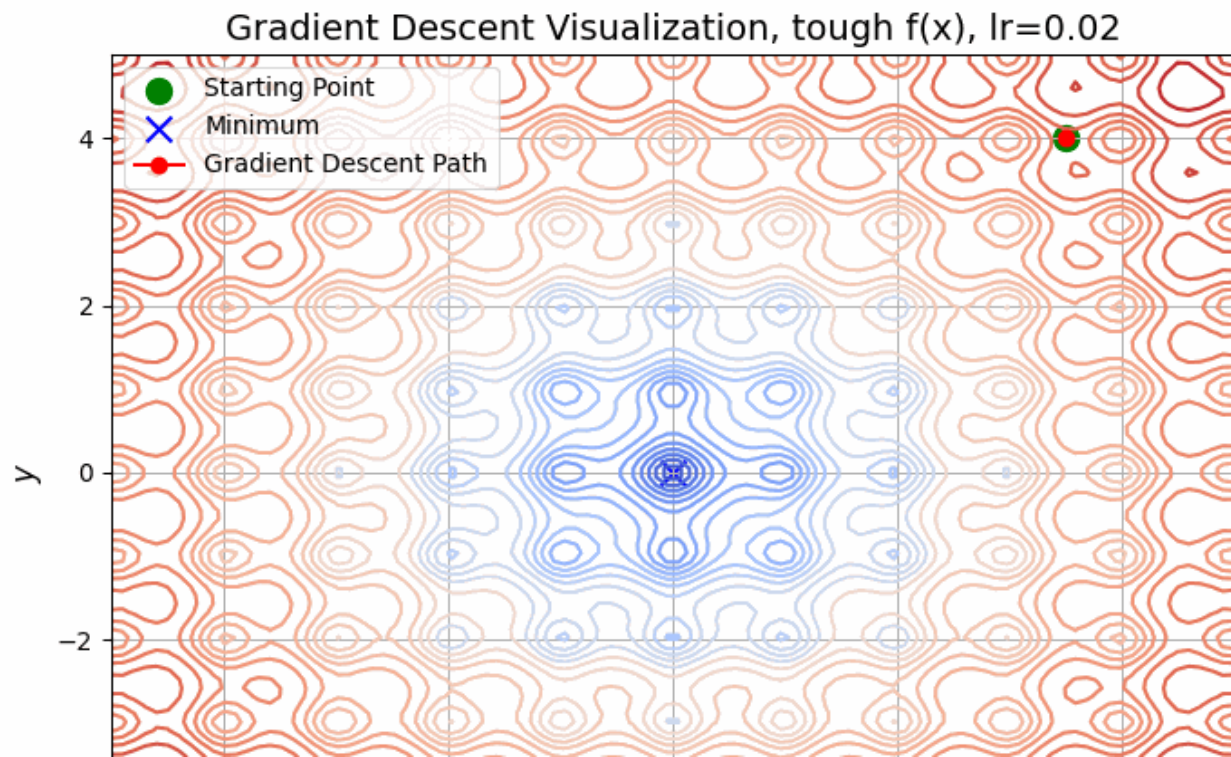
where $R^2 = |x_0 - x_*|$ and κ is the **condition number**.

3D plot of $f(x, y) = x^2 + 10y^2$

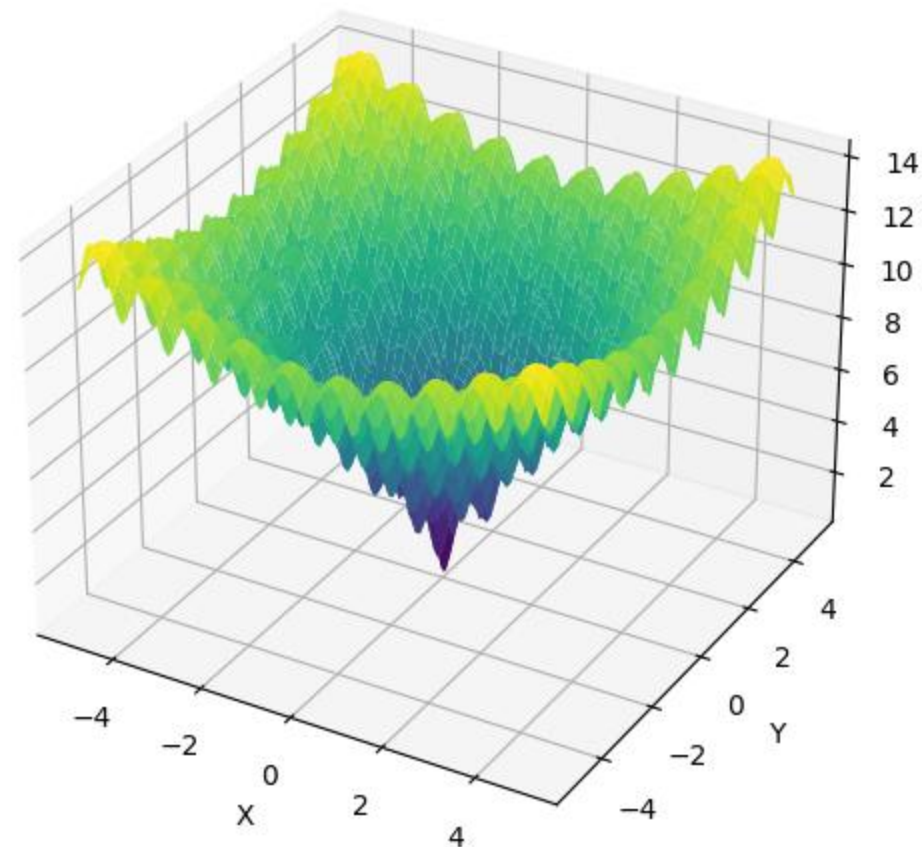


Local minima

Gradient descent may get stuck in local optima



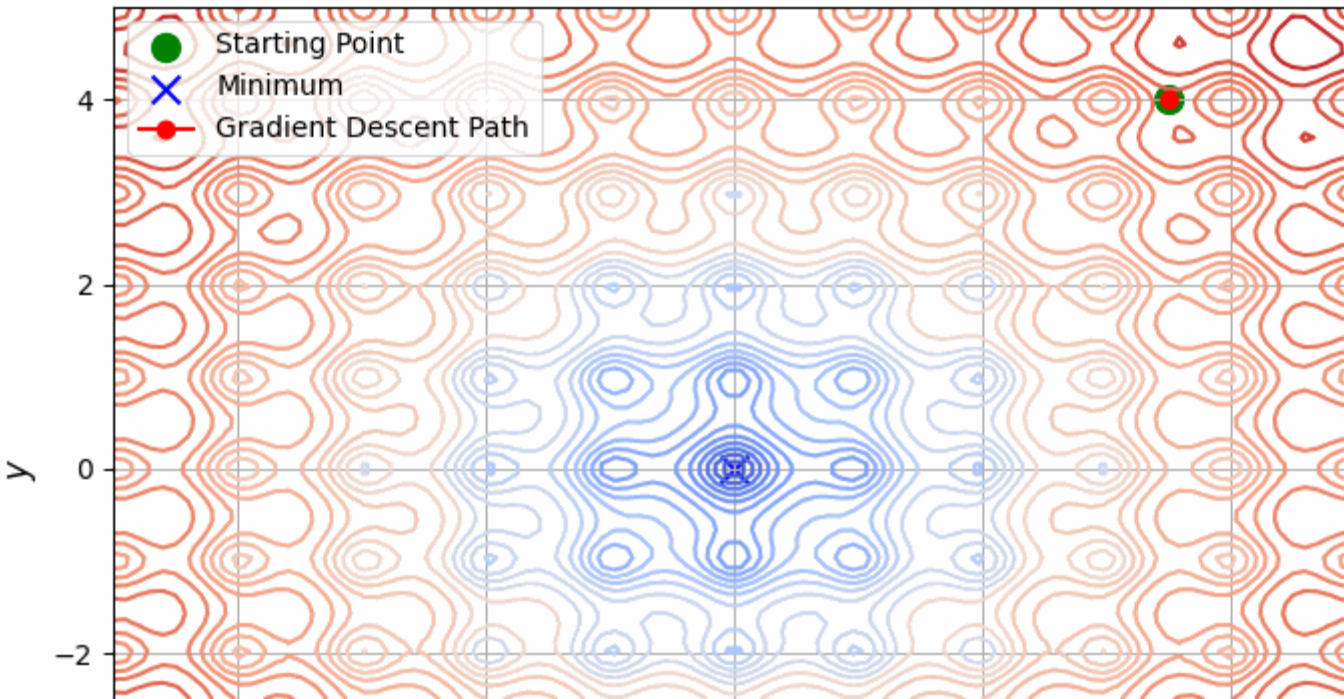
3D plot of the Ackley function



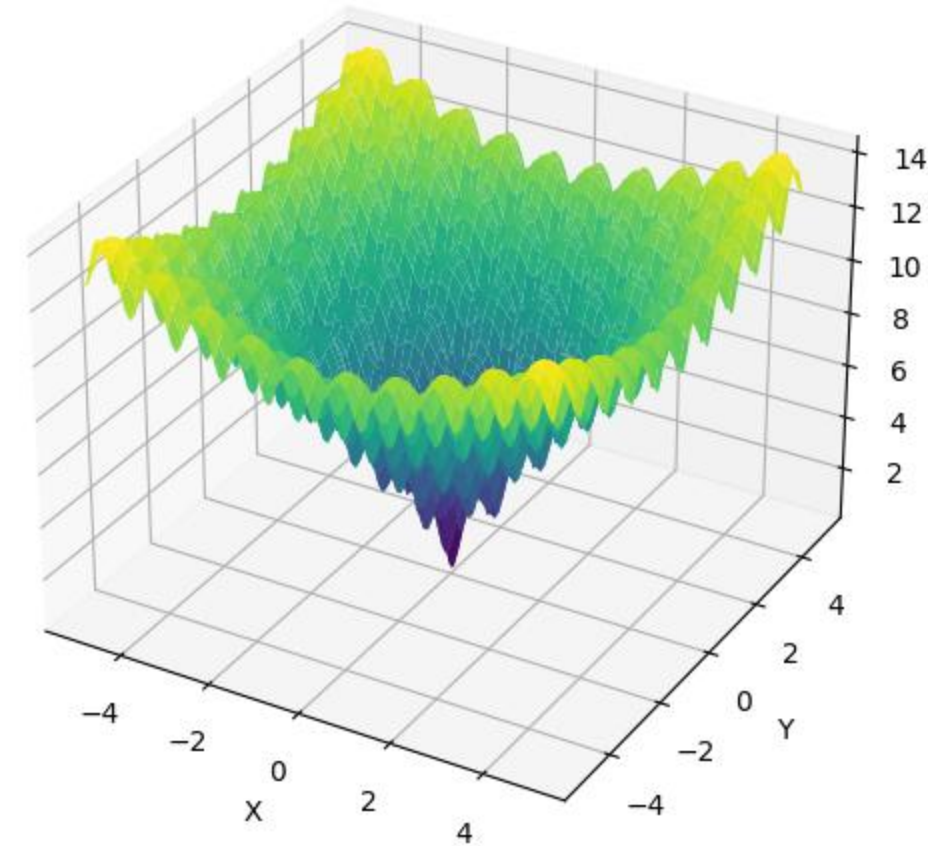
Local minima

Gradient descent may get stuck in local optima

Gradient Descent Visualization, tough $f(x)$, $lr=0.1$



3D plot of the Ackley function



Gradient descent in Machine Learning

$$\mathcal{L}(y, X, \mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y_i, x_i, \mathbf{w}) \quad \rightsquigarrow$$

$$\nabla_{\mathbf{w}} \mathcal{L} = \frac{1}{N} \sum_{i=1}^N \nabla_{\mathbf{w}} \mathcal{L}(y_i, x_i, \mathbf{w}) = \frac{1}{N} (-2y^T X + 2\mathbf{w} X^T X)$$

See any problems?

Gradient descent in Machine Learning

Why gradient descent isn't used:

- Avoid manifesting large matrices in the memory
- If your dataset is huge, we don't feedback from the system anytime soon.

Stochastic gradient
descent

Stochastic gradient descent

GD

$$w_{k+1} = w_k - \alpha \sum_{i=1}^N \nabla_w \mathcal{L}(y_i, x_i, w_k)$$

SGD with **batch size B**

Pick B **random** data points $(x_{i_s}, y_{i_s})_{s=1}^B$,

$$w_{k+1} = w_k - \alpha \sum_{s=1}^B \nabla_w \mathcal{L}(y_{i_s}, x_{i_s}, w_k)$$

A typical implementation

GD

$$w_{k+1} = w_k - \alpha \sum_{i=1}^N \nabla_w \mathcal{L}(y_i, x_i, w_k)$$

SGD with batch size B

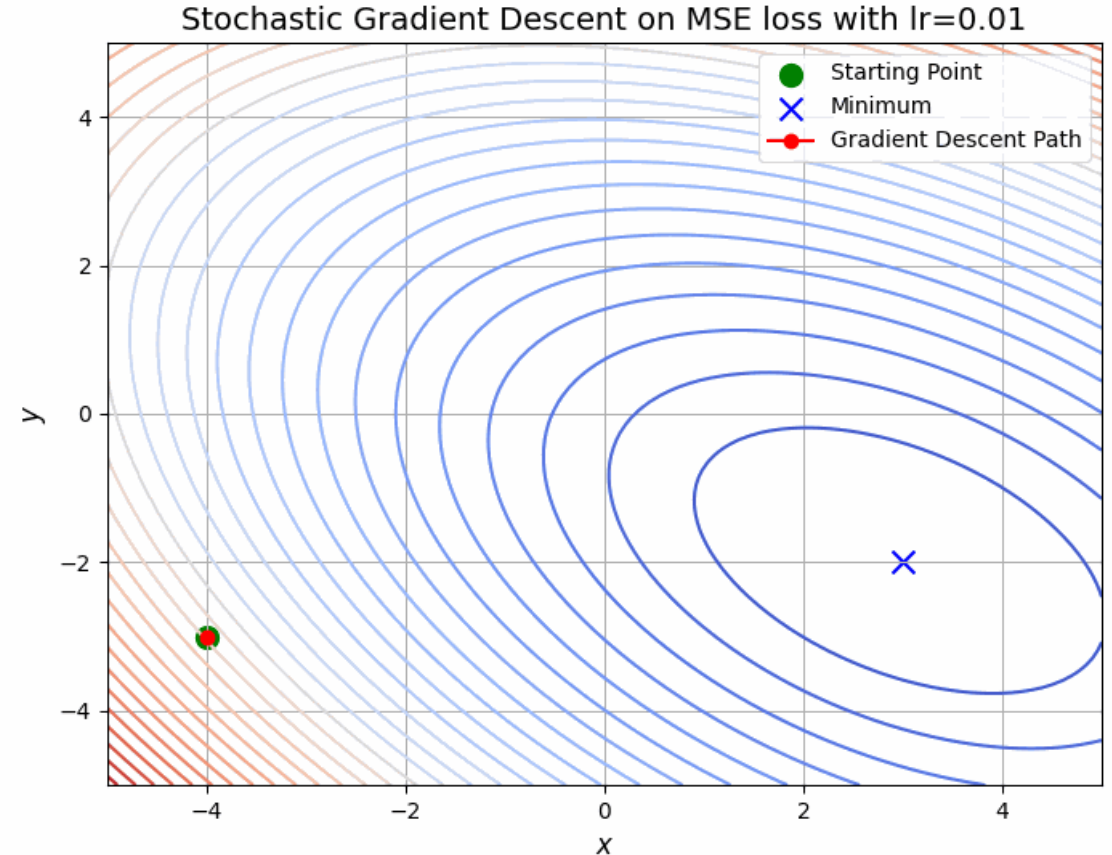
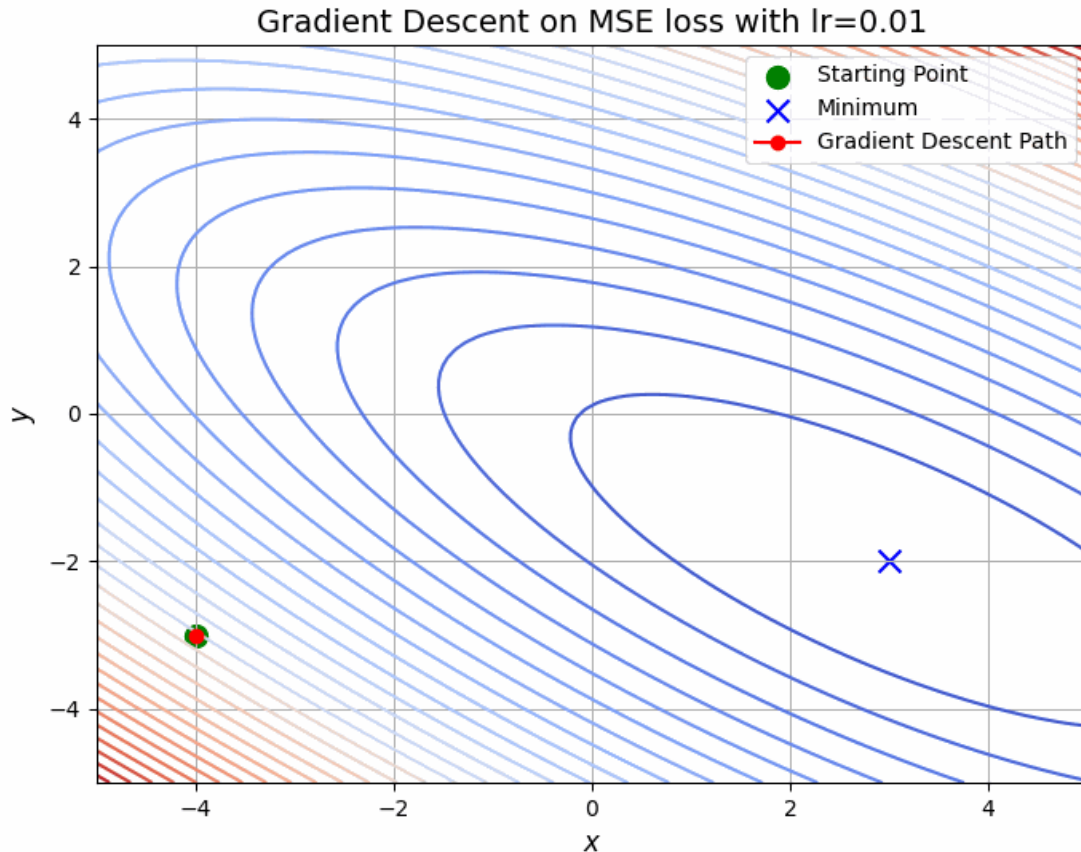
For several **epochs**:

- Randomize data order
- Now, for $t=0..(N/B-1)$

$$w_{k+1} = w_k - \alpha \sum_{i=Bt}^{Bt+B-1} \nabla_w \mathcal{L}(y_i, x_i, w_i)$$

Comparing behaviour: GD vs SGD

SGD is erratic, and the smaller the batch size, the worse



A tricky question

GD

$$w_{k+1} = w_k - \frac{\alpha}{N} \sum_{i=1}^N \nabla_w \mathcal{L}(y_i, x_i, w_k)$$

SGD with batch size B

Pick B **random** data points $(x_{i_s}, y_{i_s})_{s=1}^B$,

$$w_{k+1} = w_k - \frac{\alpha}{\text{???}} \sum_{s=1}^B \nabla_w \mathcal{L}(y_{i_s}, x_{i_s}, w_k)$$

A tricky question

GD	SGD with batch size B
$w_{k+1} = w_k - \frac{\alpha}{N} \sum_{i=1}^N \nabla_w \mathcal{L}(y_i, x_i, w_k)$	Pick B random data points $(x_{i_s}, y_{i_s})_{s=1}^B$, $w_{k+1} = w_k - \frac{\alpha}{\text{???}} \sum_{s=1}^B \nabla_w \mathcal{L}(y_{i_s}, x_{i_s}, w_k)$
$\nabla_w \mathcal{L}(y, X, w) = \frac{1}{N} \sum_{i=1}^N \nabla_w \mathcal{L}(y_i, x_i, w_k)$	$\nabla_w \mathcal{L}(y, X, w) \approx \frac{1}{B} \sum_{s=1}^B \nabla_w \mathcal{L}(y_{i_s}, x_{i_s}, w_k)$ A Monte Carlo estimate

A tricky question

GD	SGD with batch size B
$w_{k+1} = w_k - \frac{\alpha}{N} \sum_{i=1}^N \nabla_w \mathcal{L}(y_i, x_i, w_k)$	Pick B random data points $(x_{i_s}, y_{i_s})_{s=1}^B$, $w_{k+1} = w_k - \frac{\alpha}{B} \sum_{s=1}^B \nabla_w \mathcal{L}(y_{i_s}, x_{i_s}, w_k)$
$\nabla_w \mathcal{L}(y, X, w) = \frac{1}{N} \sum_{i=1}^N \nabla_w \mathcal{L}(y_i, x_i, w_k)$	$\nabla_w \mathcal{L}(y, X, w) \approx \frac{1}{B} \sum_{s=1}^B \nabla_w \mathcal{L}(y_{i_s}, x_{i_s}, w_k)$ A Monte Carlo estimate

Modifications of SGD

(Stochastic) Gradient Descent

$$g_k = \nabla_w \mathcal{L}(w_k)$$

$$w_k = w_{k-1} - \alpha_k \cdot g_k$$

Momentum

$$g_k = \nabla_w \mathcal{L}(w_k)$$

$$\mu_k = \beta_1 \cdot \mu_{k-1} + (1 - \beta_1) g_k$$

← Accumulates previous directions
Makes GD less erratic

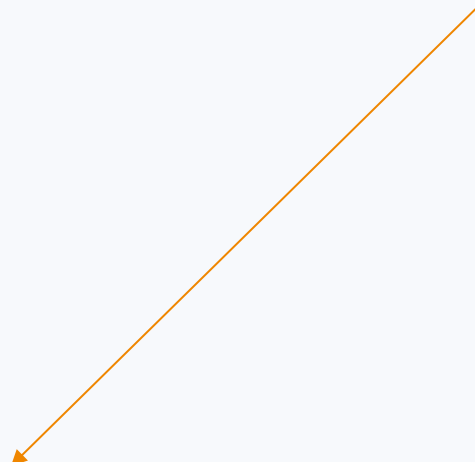
$$w_k = w_{k-1} - \alpha_k \cdot \mu_k$$

AdaGrad

$$g_k = \nabla_w \mathcal{L}(w_k)$$

$$w_k = w_{k-1} - \alpha_k \cdot \frac{g_k}{\sqrt{g_k^2 + \epsilon}}$$

Adaptive step
for each coordinate
(~second-order methods)



RMSProp

$$g_k = \nabla_w \mathcal{L}(w_k)$$

$$v_k = \beta_2 \cdot v_{k-1} + (1 - \beta_2) g_k^2$$

Adaptive step
for each coordinate
(~second-order methods)



$$w_k = w_{k-1} - \alpha_k \cdot \frac{g_k}{\sqrt{v_k + \epsilon}}$$

Bringing everything together: almost Adam

$$g_k = \nabla_w \mathcal{L}(w_k)$$

$$\mu_k = \beta_1 \cdot \mu_{k-1} + (1 - \beta_1) g_k$$

← Accumulates previous directions
Makes GD less erratic

$$v_k = \beta_2 \cdot v_{k-1} + (1 - \beta_2) g_k^2$$

← Adaptive step
for each coordinate
(~second-order methods)

$$w_k = w_{k-1} - \alpha_k \cdot \frac{\mu_k}{\sqrt{\hat{v}_k + \epsilon}}$$

Bringing everything together: Adam

$$g_k = \nabla_w \mathcal{L}(w_k)$$

$$\mu_k = \beta_1 \cdot \mu_{k-1} + (1 - \beta_1) g_k$$

← Accumulates previous directions
Makes GD less erratic

$$v_k = \beta_2 \cdot v_{k-1} + (1 - \beta_2) g_k^2$$

← Adaptive step
for each coordinate
(~second-order methods)

$$\hat{\mu}_k = \frac{\mu_k}{1 - \beta_1^k},$$

$$\hat{v}_k = \frac{v_k}{1 - \beta_2^k}$$

← Bias correction

$$w_k = w_{k-1} - \alpha_k \cdot \frac{\hat{\mu}_k}{\sqrt{\hat{v}_k + \epsilon}}$$

Adam with L_2 -regularization

$$g_k = \nabla_w \mathcal{L}(w_k) + \lambda w_k$$

$$\mu_k = \beta_1 \cdot \mu_{k-1} + (1 - \beta_1) g_k$$

Accumulates previous directions
Makes GD less erratic

$$v_k = \beta_2 \cdot v_{k-1} + (1 - \beta_2) g_k^2$$

Adaptive step
for each coordinate
(~second-order methods)

$$\hat{\mu}_k = \frac{\mu_k}{1 - \beta_1^k},$$

$$\hat{v}_k = \frac{v_k}{1 - \beta_2^k}$$

Bias correction

$$w_k = w_{k-1} - \alpha_k \cdot \frac{\hat{\mu}_k}{\sqrt{\hat{v}_k + \epsilon}}$$

AdamW: Adam with L_2 weight decay

$$g_k = \nabla_w \mathcal{L}(w_k)$$

$$\mu_k = \beta_1 \cdot \mu_{k-1} + (1 - \beta_1) g_k$$

← Accumulates previous directions
Makes GD less erratic

$$v_k = \beta_2 \cdot v_{k-1} + (1 - \beta_2) g_k^2$$

← Adaptive step
for each coordinate
(~second-order methods)

$$\hat{\mu}_k = \frac{\mu_k}{1 - \beta_1^k},$$

$$\hat{v}_k = \frac{v_k}{1 - \beta_2^k}$$

← Bias correction

$$w_k = w_{k-1} - \alpha_k \cdot \left(\frac{\hat{\mu}_k}{\sqrt{\hat{v}_k + \epsilon}} + \lambda w_k \right)$$

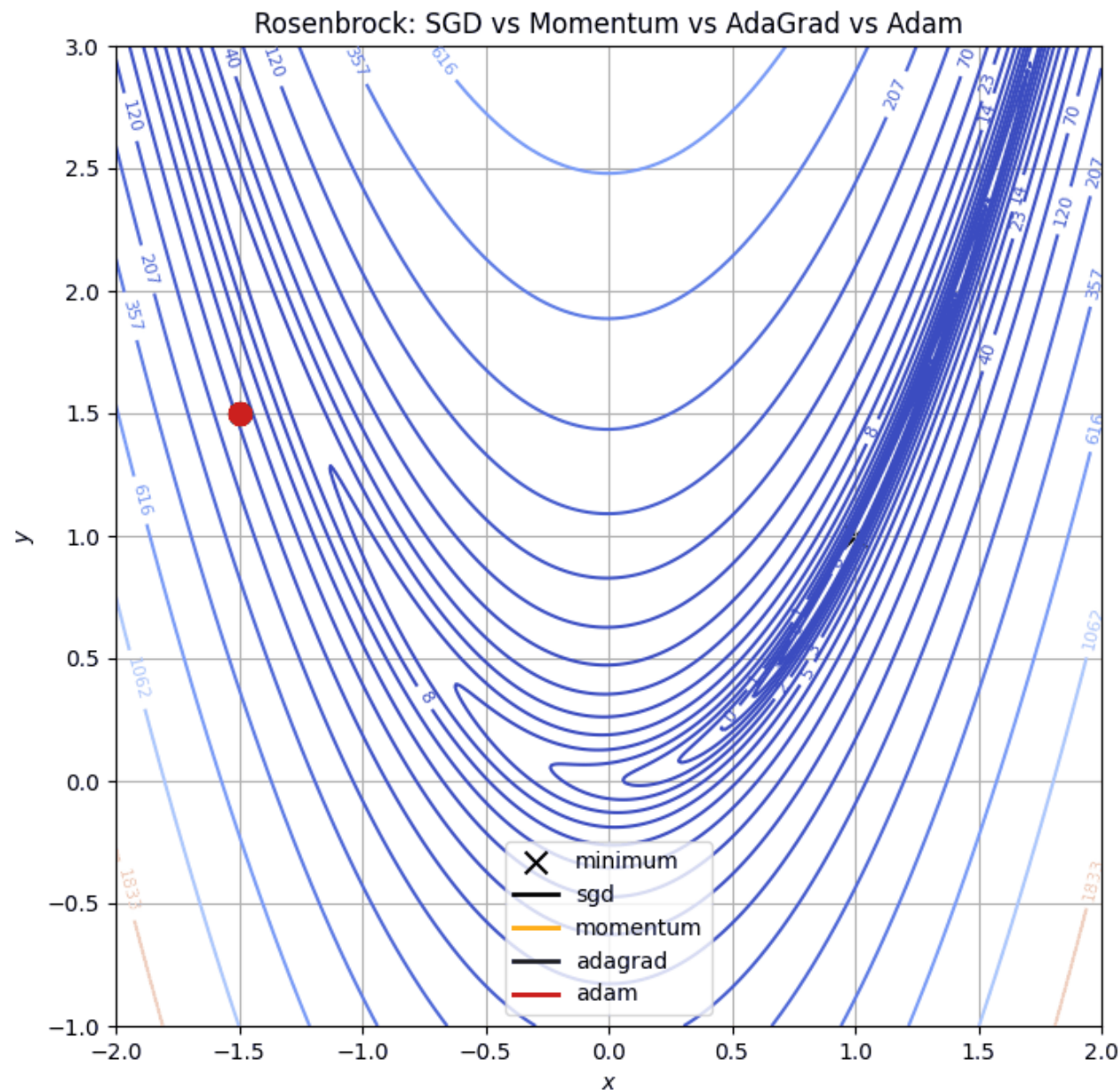
A simple experiment

Rosenbrock function

$$(1 - x)^2 + 100(y - x^2)^2$$

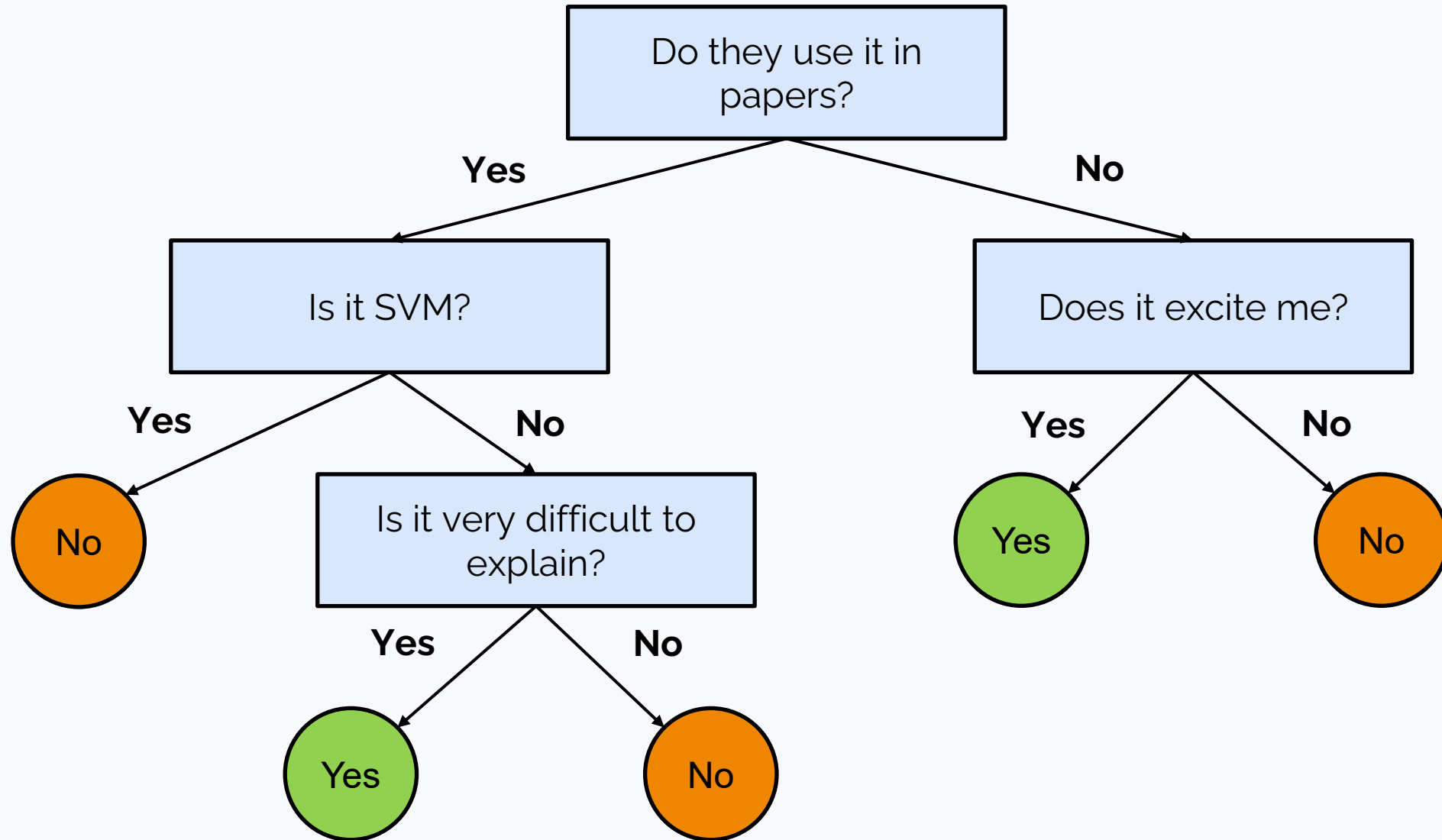
Has global minimum in

$(1, 1)$

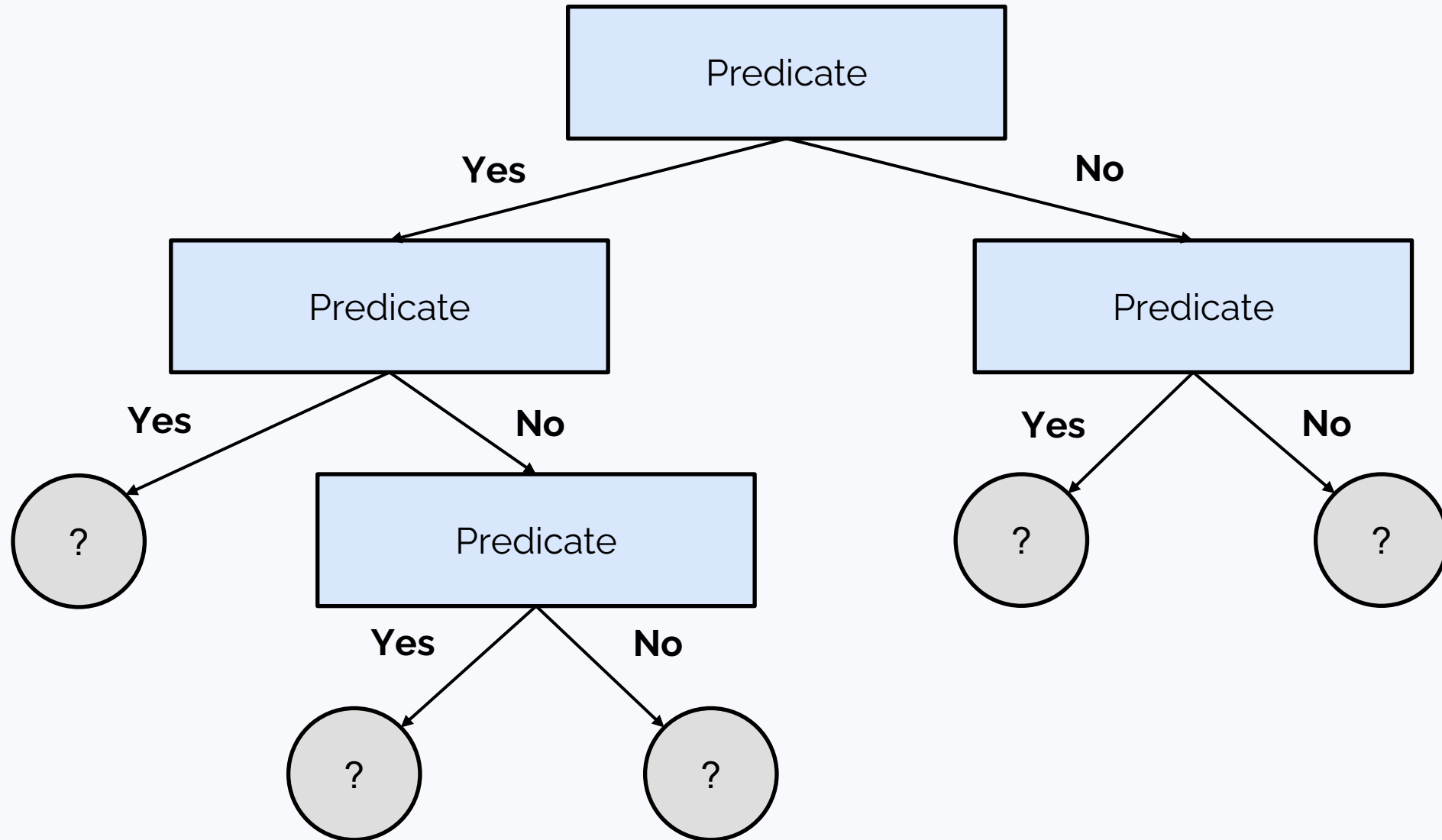


Decision trees

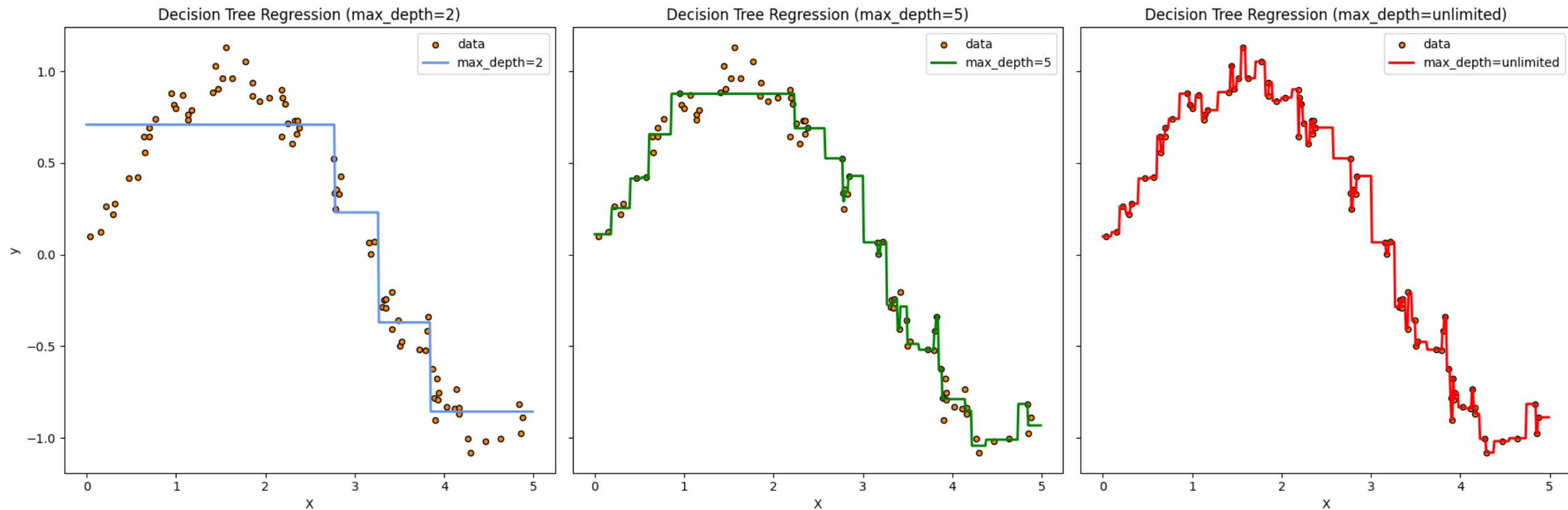
Will I feature topic X in the course?



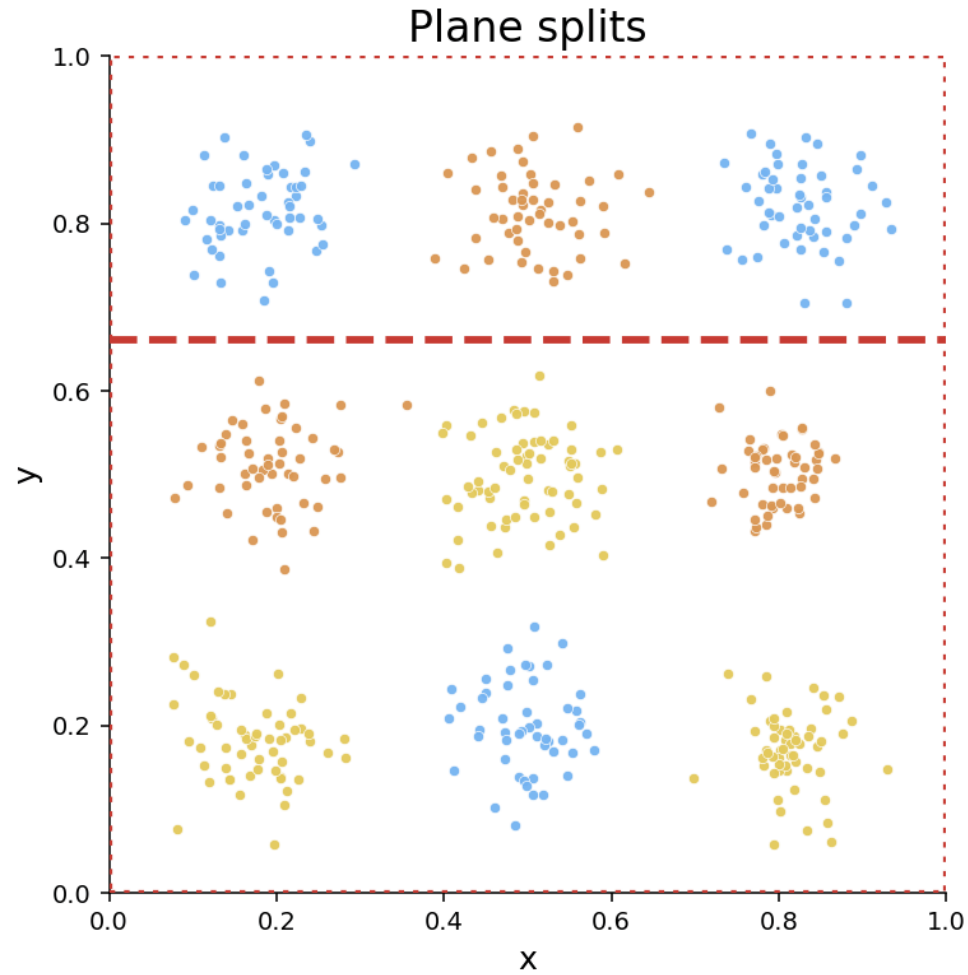
Will I feature topic X in the course?



Decision trees give piecewise linear functions



Building decision trees



Decision tree

$$y \leq 0.66$$

Building decision trees – predicates

For continuous features:

- Predicates: $\text{feature}_i \geq a_i$

Building decision trees – predicates

For continuous features:

- Predicates: $\text{feature}_i \geq a_i$

For categorical (discrete) features in case of binary classification:

- Consider $\text{feature}_i \in \mathcal{C}$ for every subset $\mathcal{C}$ of potential values!
- Sort all values c_j by $\#\{\text{class 1 objects with feature} = c_j\}$
- Take $\text{feature}_i \succ a_i$

Building decision trees – branching criteria

If we stop at X_m , all its points will get a constant value c .

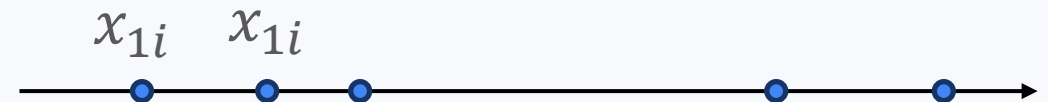
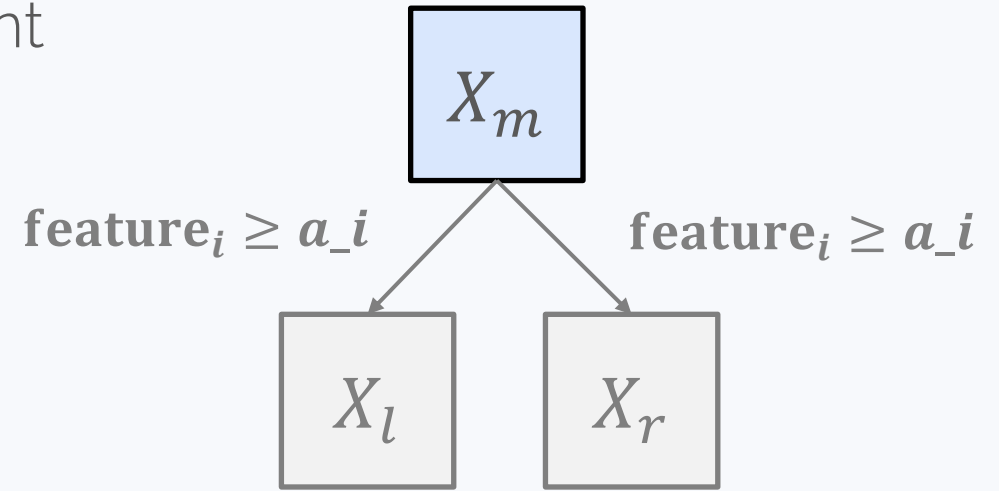
The loss will be:

$$\frac{1}{|X_m|} \sum_{(x_i, y_i) \in X_m} \mathcal{L}(y_i, c)$$

Impurity:

$$H(X_m) = \min_c \frac{1}{|X_m|} \sum_{(x_i, y_i) \in X_m} \mathcal{L}(y_i, c)$$

Compute this by exhaustive search...



Building decision trees – branching criteria

Impurity:

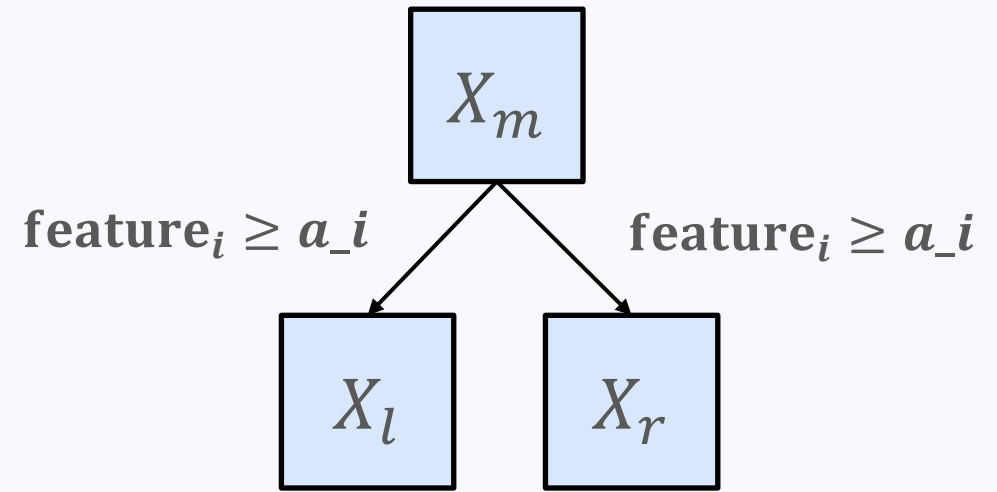
$$H(X_m) = \min_c \frac{1}{|X_m|} \sum_{(x_i, y_i) \in X_m} \mathcal{L}(y_i, c)$$

If we split, the loss is:

$$\frac{1}{|X_m|} \left(\sum_{(x_i, y_i) \in X_r} \mathcal{L}(y_i, c_r) + \sum_{(x_i, y_i) \in X_l} \mathcal{L}(y_i, c_l) \right)$$

And the impurity:

$$\frac{1}{|X_m|} \left(\min_{c_r} \sum_{(x_i, y_i) \in X_r} \mathcal{L}(y_i, c_r) + \min_{c_l} \sum_{(x_i, y_i) \in X_l} \mathcal{L}(y_i, c_l) \right) = \frac{|X_r|}{|X_m|} H(X_r) + \frac{|X_l|}{|X_m|} H(X_l)$$



Building decision trees – branching criteria

Impurity of a leaf:

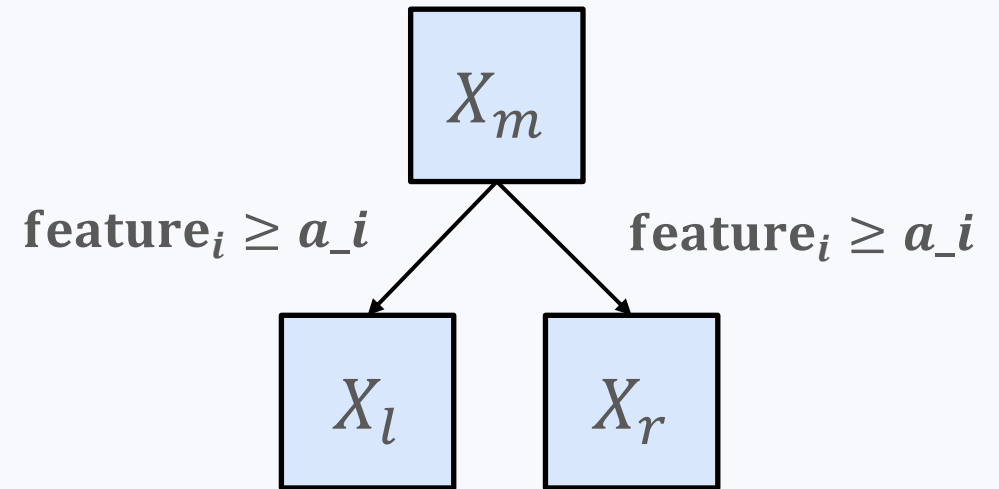
$$H(X_m) = \min_c \frac{1}{|X_m|} \sum_{(x_i, y_i) \in X_m} \mathcal{L}(y_i, c)$$

Impurity of a decision stump:

$$\frac{|X_r|}{|X_m|} H(X_r) + \frac{|X_l|}{|X_m|} H(X_l)$$

Branching criterion:

$$H(X_m) - \frac{|X_r|}{|X_m|} H(X_r) - \frac{|X_l|}{|X_m|} H(X_l)$$



Building decision trees – branching criteria

Impurity of a leaf:

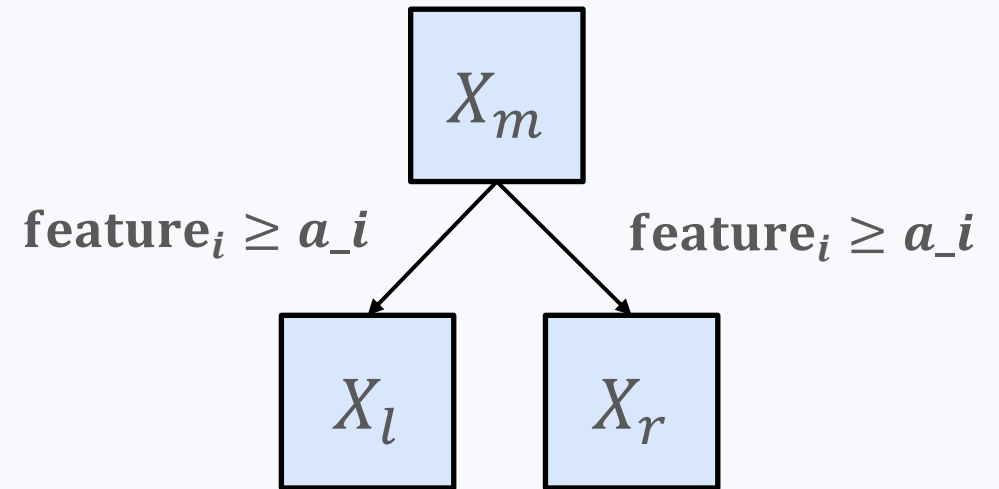
$$H(X_m) = \min_c \frac{1}{|X_m|} \sum_{(x_i, y_i) \in X_m} \mathcal{L}(y_i, c)$$

Impurity of a decision stump:

$$\frac{|X_r|}{|X_m|} H(X_r) + \frac{|X_l|}{|X_m|} H(X_l)$$

Branching criterion:

$$|X_m| \cdot H(X_m) - |X_r| \cdot H(X_r) - |X_l| \cdot H(X_l)$$

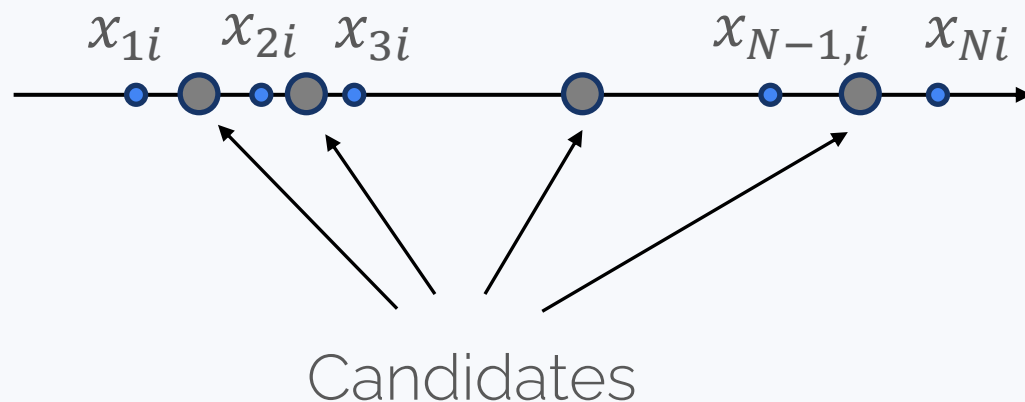


How to find the best feature and threshold?

Through exhaustive search...

First, over features.

Then:

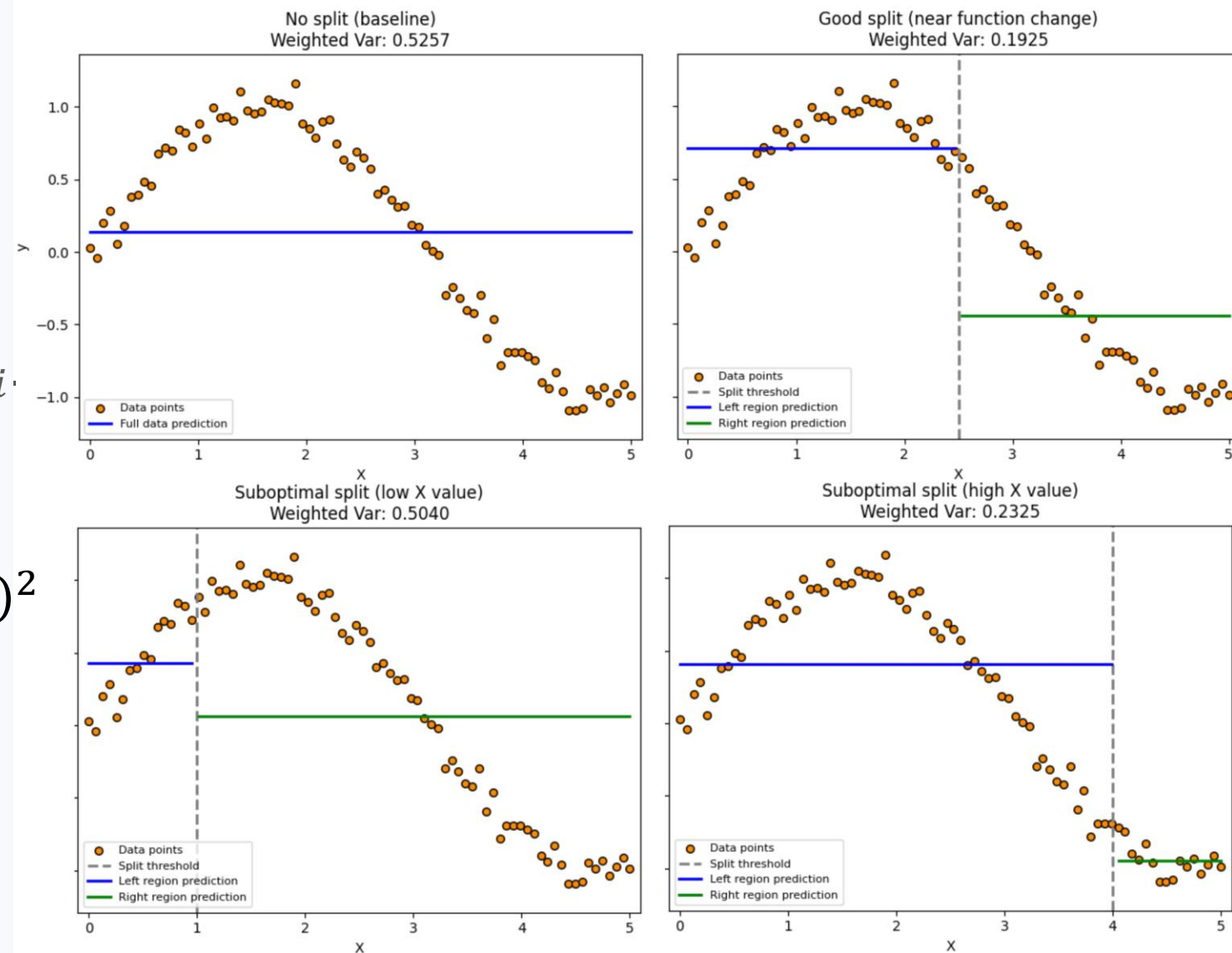


MSE regression

In a leaf, we predict one number c .
The optimal value is the mean of y_i .

Then

$$H(X_m) = \min_c \frac{1}{|X_m|} \sum_{(x_i, y_i) \in X_m} (y_i - c)^2$$
$$= \mathbb{V}_{X_m}(y)$$



Multiclass classification: Entropy criterion

In a leaf, we predict class probabilities: $(p_1, \dots, p_K)$.

Optimal values are:

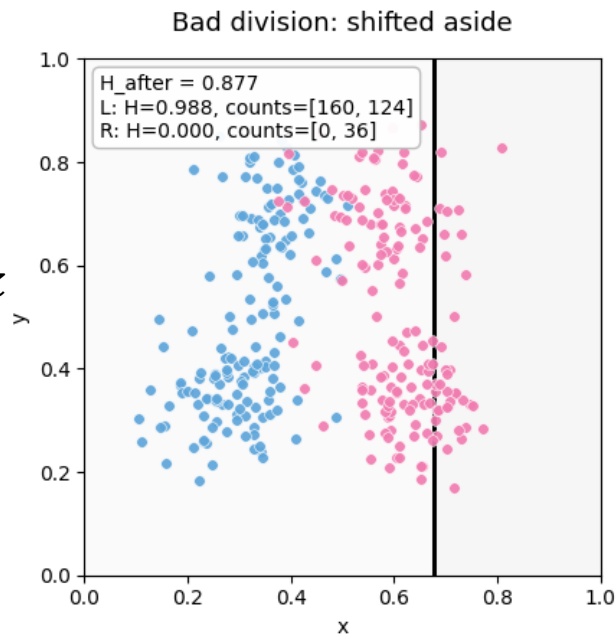
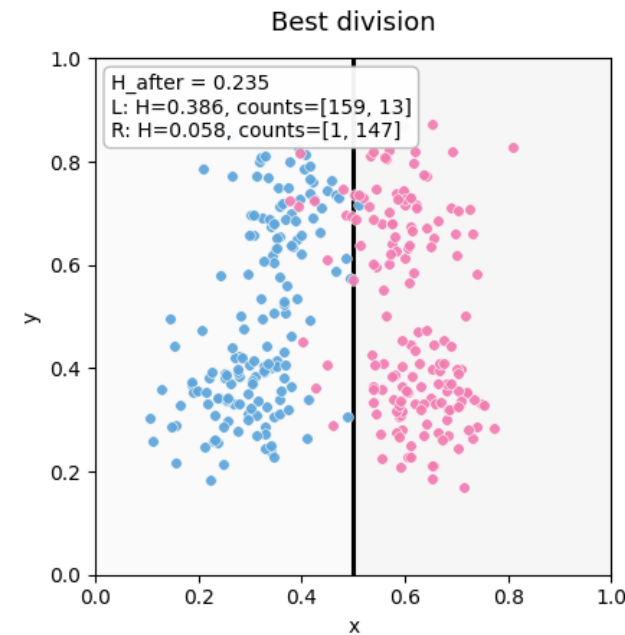
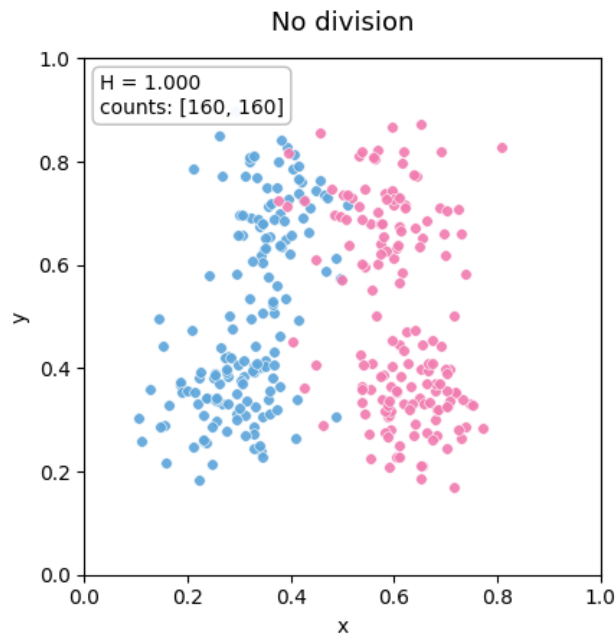
$$\hat{p}_k = \frac{\#\{x_i \text{ of class } k\}}{|X_m|}$$

Then

$$H(X_m) = \min_p \frac{1}{|X_m|} \sum_{(x_i, y_i) \in X_m} -[y_i = k] \log p_k$$

Binary dataset: entropy before and after candidate divisions

● class 0 ● class 1



Multiclass classification: Entropy criterion

In a leaf, we predict class probabilities: $(p_1, \dots, p_K)$.

Optimal values are:

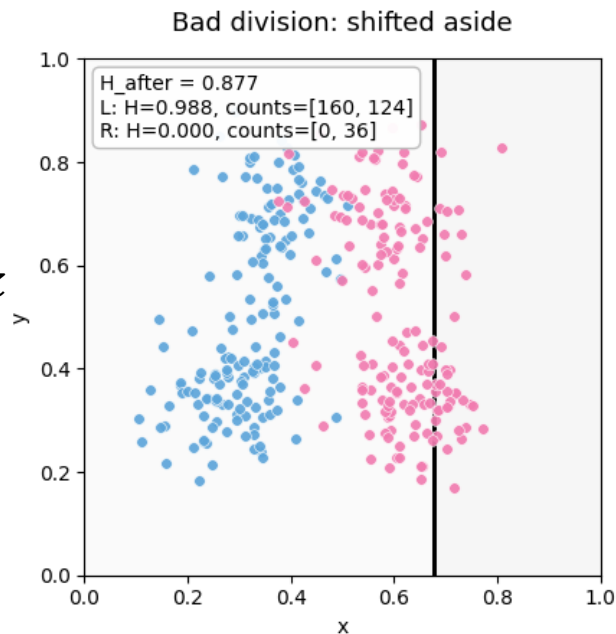
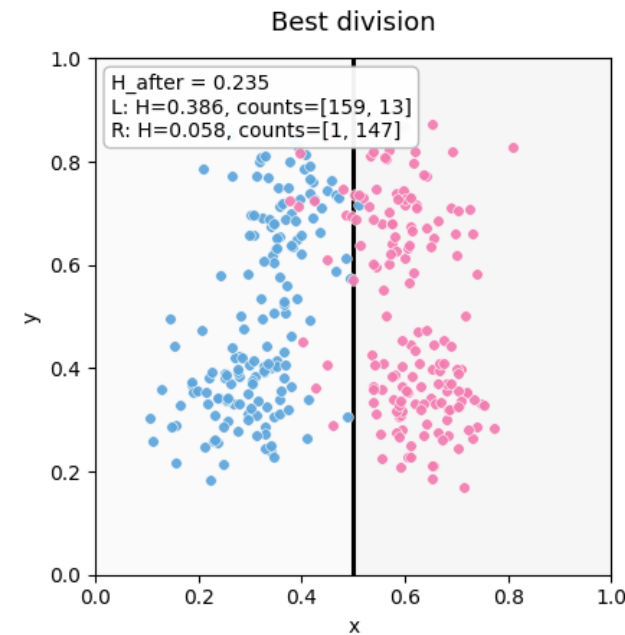
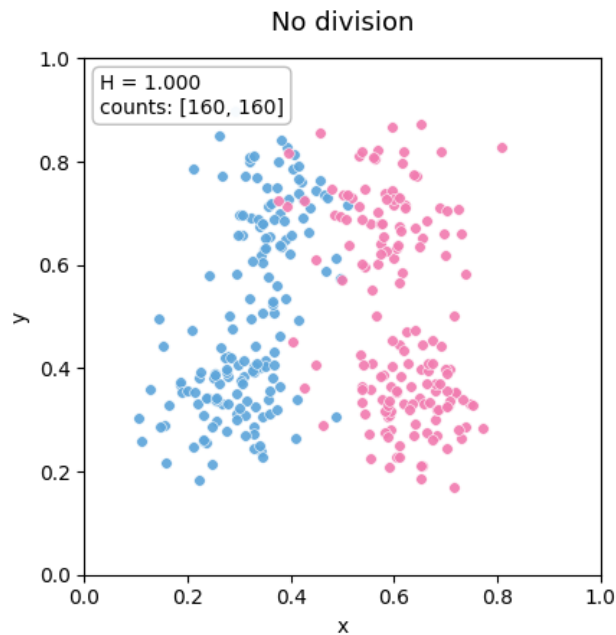
$$\hat{p}_k = \frac{\#\{x_i \text{ of class } k\}}{|X_m|}$$

Then

$$\begin{aligned} H(X_m) &= \min_p \frac{1}{|X_m|} \sum_{(x_i, y_i) \in X_m} -[y_i = k] \log p_k \\ &= - \sum_{k=1}^K \hat{p}_k \log \hat{p}_k \end{aligned}$$

Binary dataset: entropy before and after candidate divisions

● class 0 ● class 1



Multiclass classification: Gini criterion

In a leaf, we predict class probabilities: $(p_1, \dots, p_K)$.

Optimal values are:

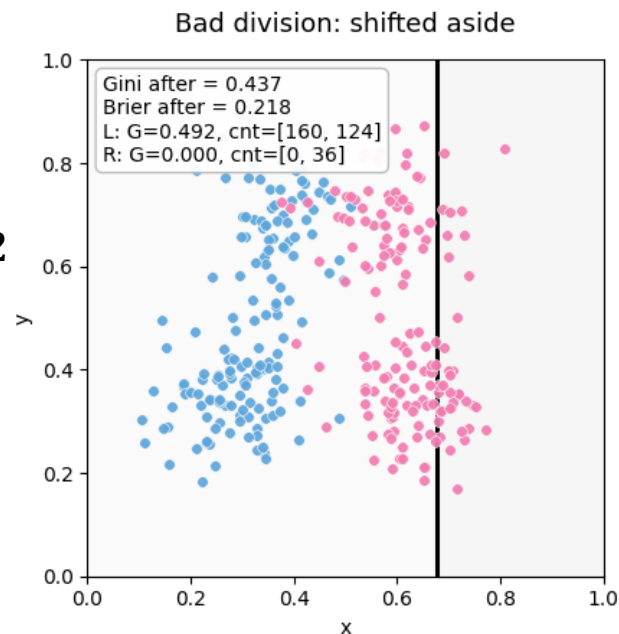
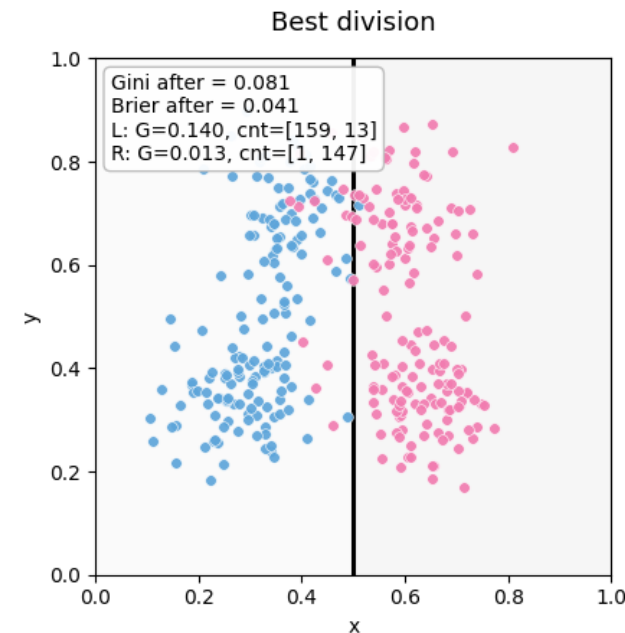
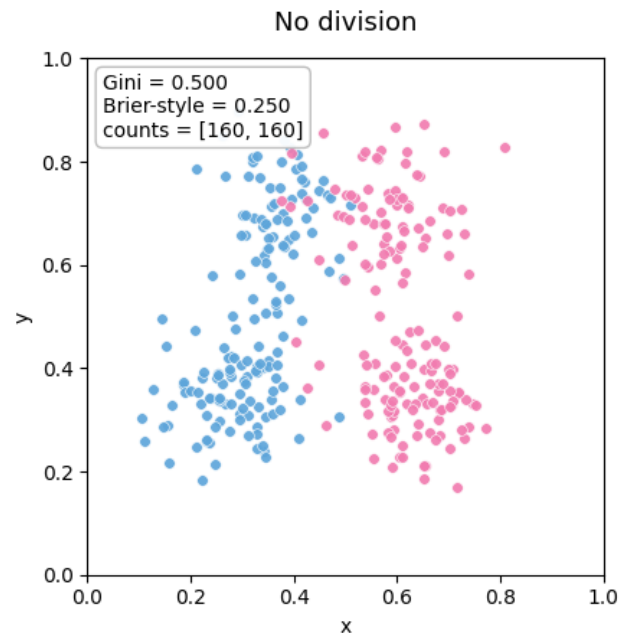
$$\hat{p}_k = \frac{\#\{x_i \text{ of class } k\}}{|X_m|}$$

Then

$$\begin{aligned} H(X_m) &= \min_p \frac{1}{|X_m|} \sum_{(x_i, y_i) \in X_m} (p_k - [y_i = k])^2 \\ &= - \sum_{k=1}^K \hat{p}_k (1 - \hat{p}_k) \end{aligned}$$

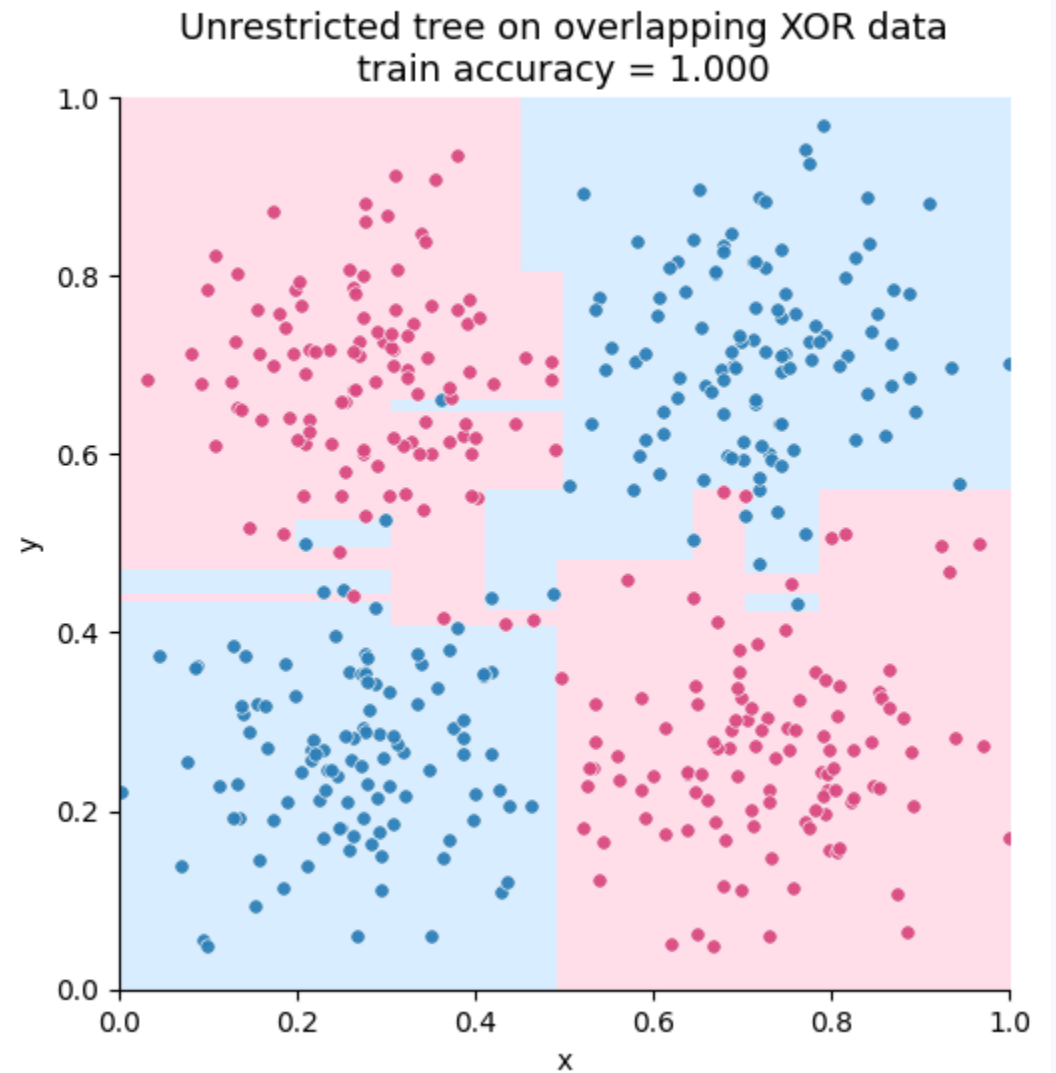
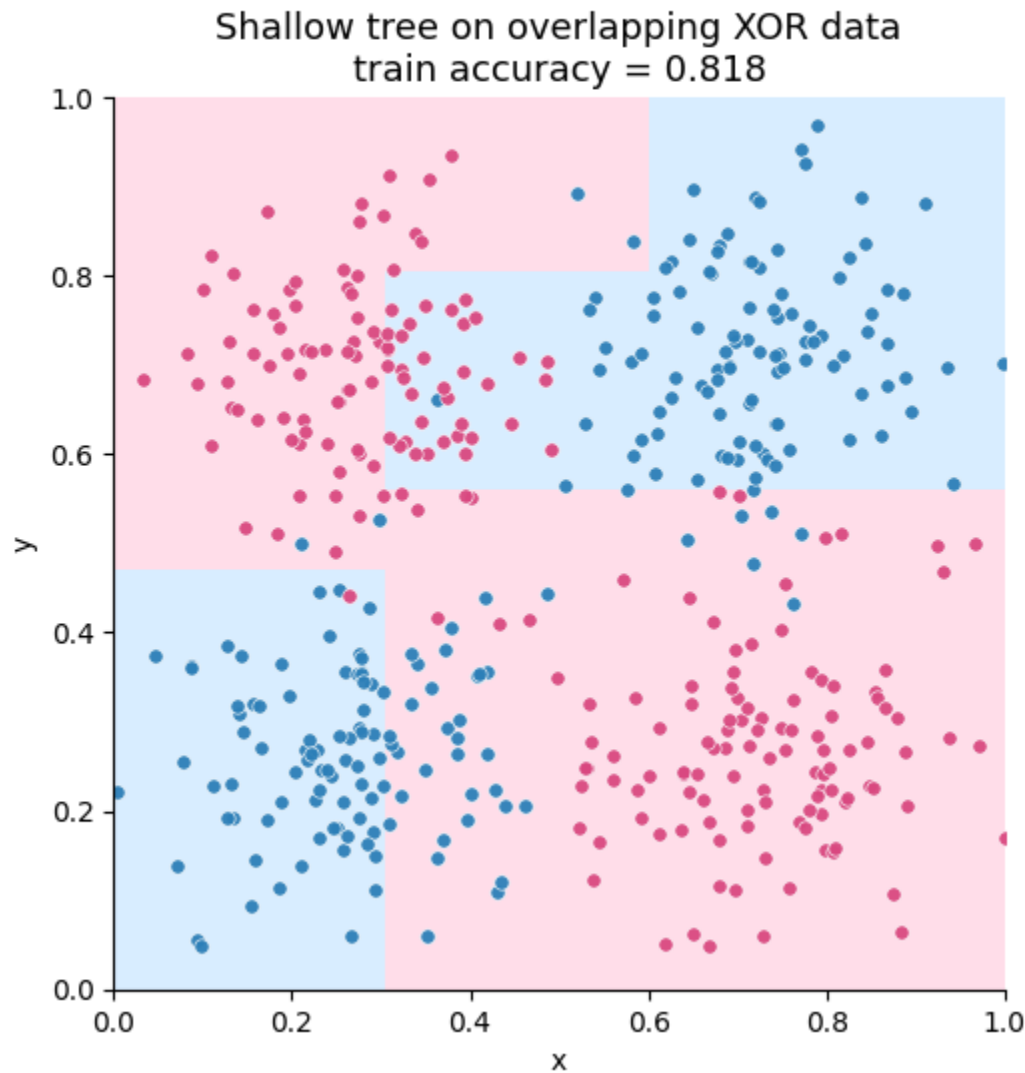
Binary dataset: Gini / Brier-style impurity before and after candidate divisions

● class 0 ● class 1



Decision trees are champions at overfitting

● class 0 ● class 1



Decision trees are champions at overfitting

What to do? :O

- Restrict the tree's depths (you only need level 1-2 predicates for an exploration task)
max_depth parameter
- Check that the nodes don't contain too few points
min_samples_leaf parameter

Machine-Learning Number Fields

Numerical field extensions

Take $\mathbb{F} = \mathbb{Q}(\zeta_7) \supseteq \mathbb{Q}$, where $\zeta_7 = \exp\left(\frac{2\pi i}{7}\right)$.

Minimal polynomial of ζ_7 : $\Phi_7(x) = x^6 + x^5 + x^4 + x^3 + x^2 + x + 1$

Extension degree: $[\mathbb{F} : \mathbb{Q}] = \dim_{\mathbb{Q}} \mathbb{F} = 6$.

Automorphism group: $\sigma: \mathbb{F} \rightarrow \mathbb{F}$ is defined by $\sigma(\zeta_7) = \exp\left(\frac{2\pi ki}{7}\right) = \zeta_7^k$.

$$\text{Gal}(\mathbb{F}/\mathbb{Q}) \cong (\mathbb{Z}/7\mathbb{Z})^\times$$

$\# \text{Gal}(\mathbb{F}/\mathbb{Q}) = [\mathbb{F} : \mathbb{Q}] \Rightarrow \mathbb{F}/\mathbb{Q}$ is Galois.

Find lots of info about this field here:

<https://www.lmfdb.org/NumberField/6.0.16807.1>

Zeta function

Take $\mathbb{F} = \mathbb{Q}(\zeta_7) \supseteq \mathbb{Q}$, where $\zeta_7 = \exp\left(\frac{2\pi i}{7}\right)$.

Ordinary zeta function:

$$\zeta_{\mathbb{Q}}(s) = \sum_{n \geq 1} \frac{1}{n^s} = 1 + \frac{1}{2^s} + \frac{1}{3^s} + \dots$$

Zeta function

Take $\mathbb{F} = \mathbb{Q}(\zeta_7) \supseteq \mathbb{Q}$, where $\zeta_7 = \exp\left(\frac{2\pi i}{7}\right)$.

Ordinary zeta function:

$$\zeta_{\mathbb{Q}}(s) = \sum_{n \geq 1} \frac{1}{n^s} = 1 + \frac{1}{2^s} + \frac{1}{3^s} + \dots$$

Ring of integers $\mathcal{O}_{\mathbb{Q}}$ is the set of roots in $\mathbb{Q}$ of polynomials $x^m + a_{m-1}x^{m-1} + \dots + a_1x + a_0, a_i \in \mathbb{Z}$

Easy to see: $\mathcal{O}_{\mathbb{Q}} = \mathbb{Z}$.

Ideals in $\mathbb{Z}$ are $I = n\mathbb{Z}$, and $|\mathbb{Z}/I| = n$.

$$\zeta_{\mathbb{Q}}(s) = \sum_{\mathcal{O}_{\mathbb{Q}} \supset I \neq 0} \frac{1}{N(I)^s}, \quad \text{where } N(I) = |\mathcal{O}_{\mathbb{Q}}/I|$$

Zeta function

Take $\mathbb{F} = \mathbb{Q}(\zeta_7) \supseteq \mathbb{Q}$, where $\zeta_7 = \exp\left(\frac{2\pi i}{7}\right)$.

Zeta function for $\mathbb{F}$:

$$\zeta_{\mathbb{F}}(s) = \sum_{\mathcal{O}_{\mathbb{F}} \supset I \neq 0} \frac{1}{N(I)^s}, \quad \text{where } N(I) = |\mathcal{O}_{\mathbb{F}}/I|$$

$\mathcal{O}_{\mathbb{F}}$ is the set of roots in $\mathbb{F}$ of polynomials $x^m + a_{m-1}x^{m-1} + \cdots + a_1x + a_0, a_i \in \mathbb{Z}$

Theorem: $\mathcal{O}_{\mathbb{Q}} = \mathbb{Z}[\zeta_7]$.

Zeta function can be rewritten as:

$$\zeta_{\mathbb{F}}(s) = \sum_n \frac{a_n}{n^s}, \quad \text{where } a_n = \#\{N(I) = n\}$$

It is efficiently computed by Sage by decomposing into prime ideals.

ML experiments

Take $\mathbb{F} = \mathbb{Q}(\zeta_7) \supseteq \mathbb{Q}$, where $\zeta_7 = \exp\left(\frac{2\pi i}{7}\right)$.

First 1000 zeta function coefficients $\rightarrow$ Galois group $\mathcal{C}_4$ vs $\mathcal{C}_6$ vs $\mathcal{C}_8$.

- **Random forest classifier** gives accuracy **0.999**

Coefficients of minimal polynomial $\rightarrow$ Galois group $\mathcal{C}_8$ vs D_4 .

- **Random forest classifier** gives accuracy **0.971**

First 1000 zeta function coefficients $\rightarrow$ Galois group $\mathcal{C}_8$ vs D_4 .

- **Random forest classifier** gives accuracy **0.973**

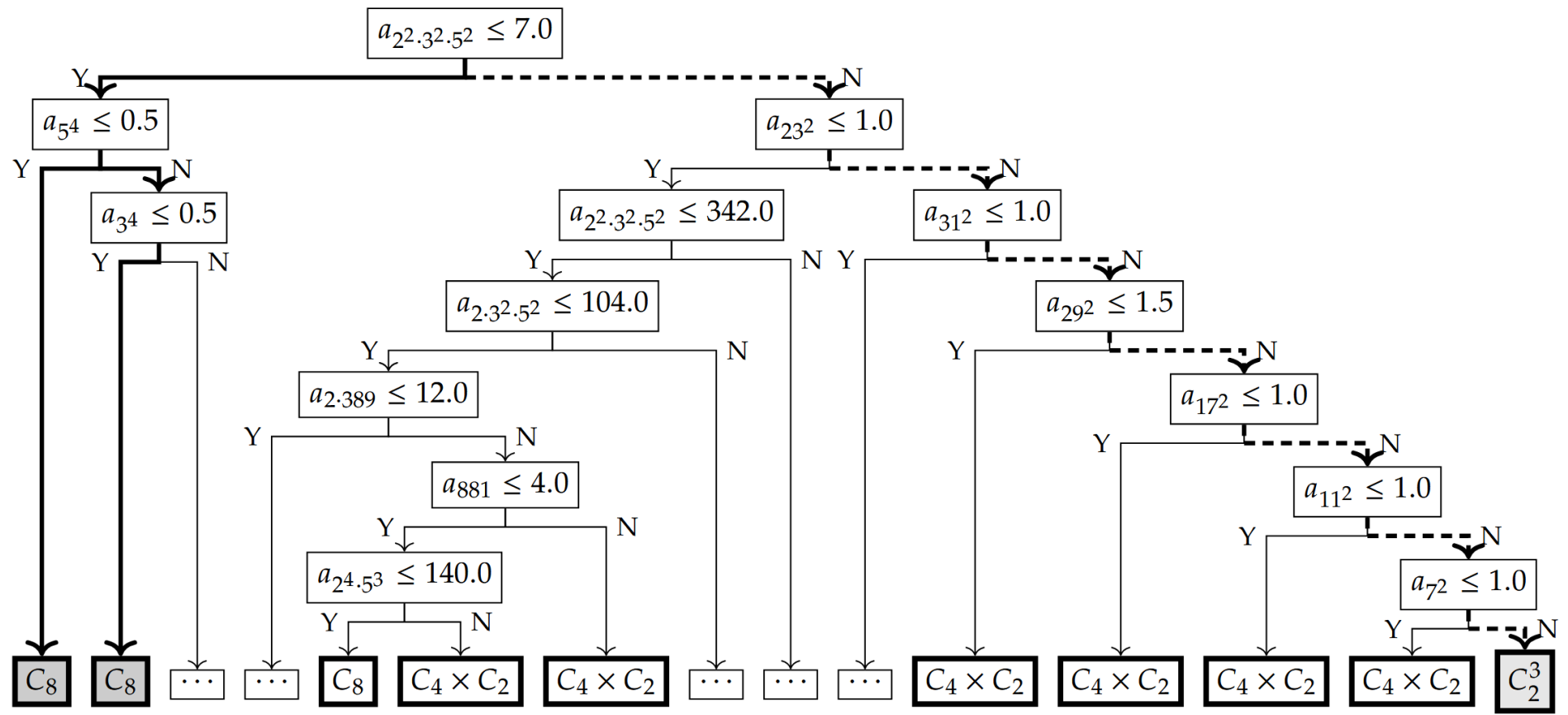
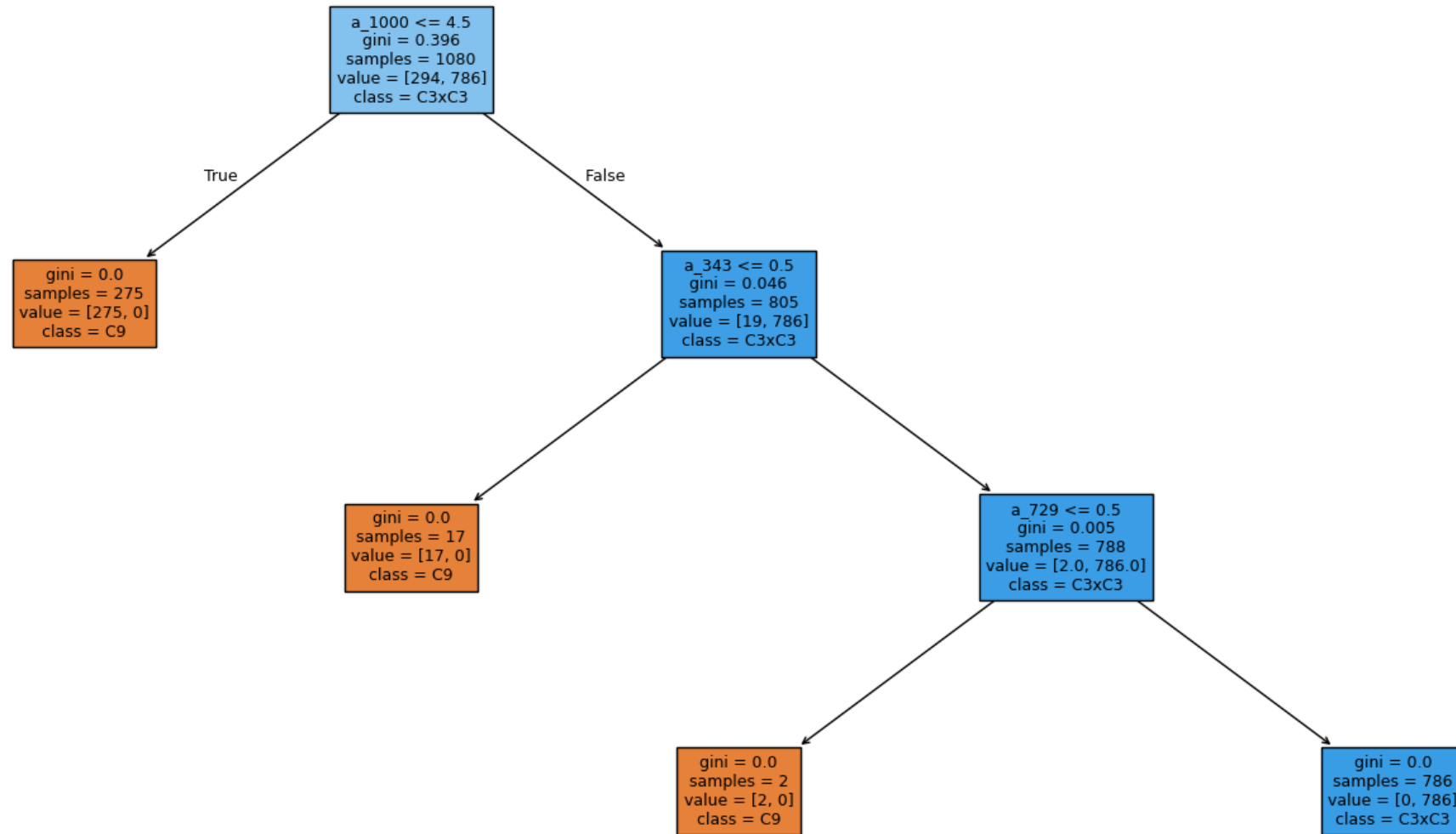


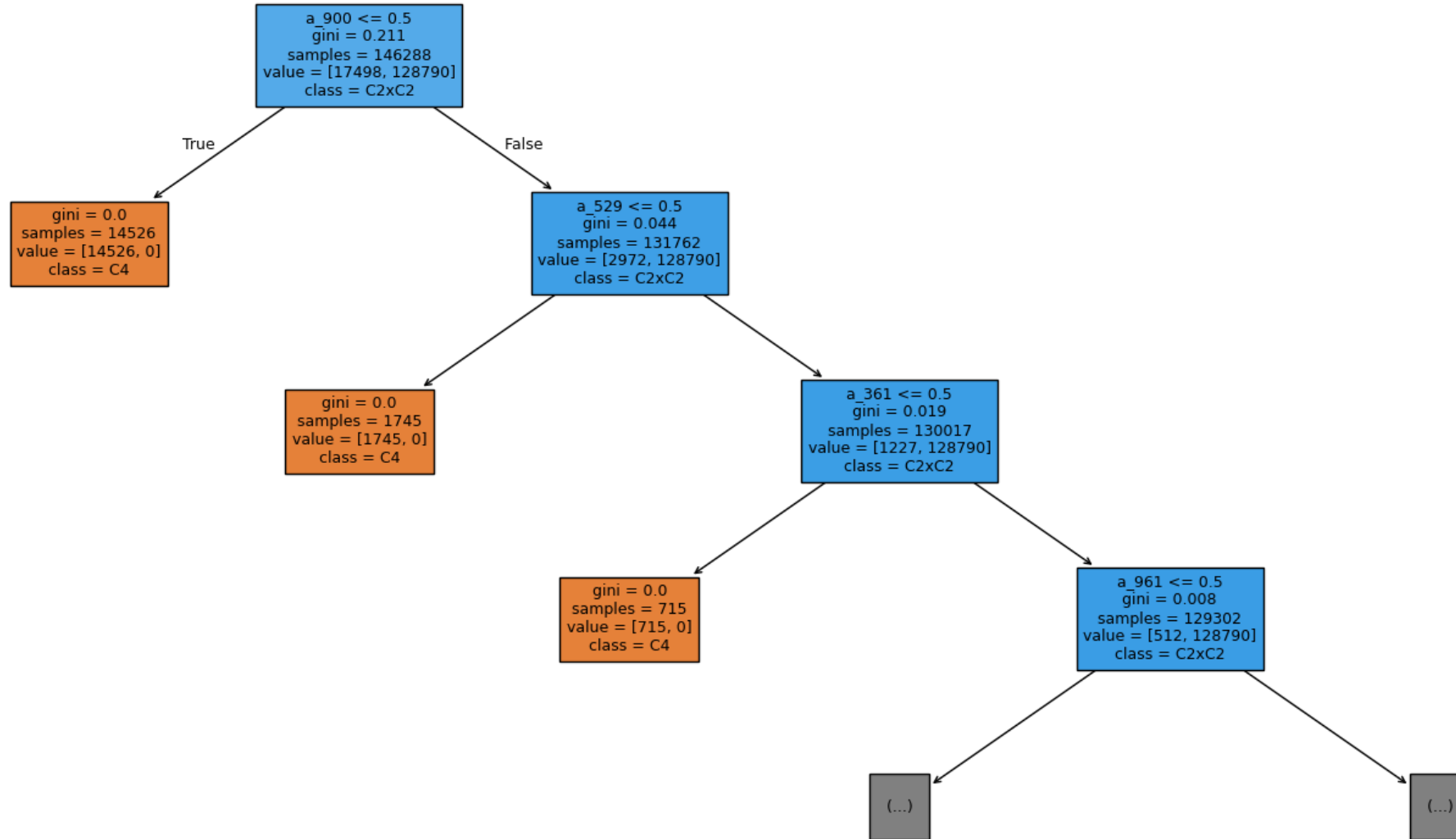
Figure 6: A decision tree predicts Galois groups of abelian octic fields from zeta coefficients up to $a_{1000}(K)$.

Theorem 3.23. Let $K/\mathbb{Q}$ be a degree ℓ^3 abelian extension with $G = \text{Gal}(K/\mathbb{Q}) \simeq C_{\ell^2} \times C_{\ell}$. Then the set of rational primes p for which $a_{p^{\ell}}(K) = 0$ has positive density. Moreover, $a_{p^{\ell^2}}(K) \neq 0$ for all primes p ; that is, the p^{ℓ^2} -th zeta coefficients are always nonzero.

C_9 VS $C_3 \times C_3$



C_4 VS $C_2 \times C_2$



C_4 VS $C_2 \times C_2$



C_4 VS $C_2 \times C_2$

